

INDIAN INSTITUTE OF TECHNOLOGY PALAKKAD

DS3020 Database Systems Final Report

Abhay Vaidya(142401005)

Jatin Vimal(142401017)

Manthan(142401027)

Punit Yadav(142401030)

Submitted to: Dr. Koninika Pal

23 April 2026



INDIAN INSTITUTE
OF TECHNOLOGY
PALAKKAD

1. INTRODUCTION

The **TradeFlow — Online Stock Trading and Portfolio Management System** is a relational database application designed to handle the end-to-end process of equity trading. It allows users to register, complete KYC verification, add funds to a wallet, place buy or sell orders, view executed trades, and monitor their portfolio in real time.

The system is built using PostgreSQL and is organized into multiple interconnected tables that manage different aspects such as user information, trading activities, wallet transactions, role-based access control, auditing, and concurrency handling.

To ensure reliability and accuracy, the database makes use of core integrity mechanisms like primary keys, foreign keys, check constraints, triggers, and stored functions. These components work together to maintain consistent data, enforce business rules, and support smooth operation of the trading workflow.

2. RELATIONAL DESIGN of the DATABASE

Each relation is described below with its attributes, primary key (PK), foreign keys (FK), and inter-table connections.

1. USERS

```
CREATE TABLE users (  
  user_id INT NOT NULL DEFAULT nextval('users_seq'),  
  name VARCHAR(100) NOT NULL,  
  email VARCHAR(100) NOT NULL,  
  kyc_status VARCHAR(10),  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  password TEXT,  
  role VARCHAR(10) DEFAULT 'USER',  
  is_active BOOLEAN DEFAULT true,  
  last_login TIMESTAMP,  
  updated_at TIMESTAMP DEFAULT NOW(),  
  
  PRIMARY KEY (user_id),  
  
  CONSTRAINT chk_kyc_status CHECK (  
    kyc_status IN ('VERIFIED', 'PENDING', 'REJECTED')  
  ),  
  CONSTRAINT chk_role CHECK (  
    role IN ('USER', 'ADMIN', 'TRADER', 'ANALYST')  
  )  
);
```

data_id	name	email	kyc_status	created_at	password	role	is_active	last_login	updated_at
(PK) integer	character varying (100)	character varying (100)	character varying (10)	timestamp without time zone	text	character varying (10)	boolean	timestamp without time zone	timestamp without time zone
1	2	Angie Henderson	angieh@example.net	VERIFIED	2024-11-27 07:16:00	password	USER	true	[null]
2	4	Abigail Shaffer	abigailsh@example.org	VERIFIED	2024-06-01 13:24:00	password	USER	true	[null]
3	6	Lindsay Roman	lindsayr@example.org	VERIFIED	2024-12-13 06:24:00	password	ADMIN	true	[null]
4	8	Laura Ruth	laurar@example.org	VERIFIED	2024-03-19 15:49:00	password	USER	true	[null]
5	9	Sharon James	sharonj@example.org	VERIFIED	2024-03-20 02:58:00	password	USER	true	[null]
6	12	Melinda Cameron	melindac@example.com	VERIFIED	2024-01-12 05:55:00	password	USER	true	[null]
7	13	Austin Gertry	austing@example.net	VERIFIED	2024-05-15 22:58:00	password	USER	true	[null]
8	19	Prima Mahla	primam@example.com	VERIFIED	2024-12-22 04:24:00	password	USER	true	[null]
9	16	Zachary Taylor	zacharyt@example.org	VERIFIED	2024-10-21 09:00:00	password	USER	true	[null]
10	31	Sanya Johnston	sanyaj@example.org	VERIFIED	2024-13-18 12:38:00	password	USER	true	[null]
11	38	Victoria Garcia	victoriag@example.org	VERIFIED	2024-02-09 09:32:00	password	USER	true	[null]
12	40	Neda Singh	nedas@example.net	VERIFIED	2024-12-10 04:28:00	password	USER	true	[null]
13	41	Anthony Woods	anthonyw@example.org	VERIFIED	2024-07-13 23:17:00	password	USER	true	[null]
14	44	Emily Green	emilieg@example.net	VERIFIED	2024-08-08 08:30:00	password	USER	true	[null]
15	45	Rahul Sharma	rahuls@example.org	VERIFIED	2024-11-22 12:30:00	password	USER	true	[null]
16	47	Sherry Wood	sherryw@example.com	VERIFIED	2024-10-31 03:00:00	password	USER	true	[null]
17	50	Rohit Verma	rohitv@example.com	VERIFIED	2024-07-10 14:09:00	password	USER	true	[null]
18	53	Michael Hoffman	michaelh@example.org	VERIFIED	2024-01-11 20:36:00	password	USER	true	[null]
19	56	Allison Doyle	allisond@example.com	VERIFIED	2024-10-26 05:14:00	password	USER	true	[null]
20	61	Jennifer Ramirez	jenniferr@example.org	VERIFIED	2024-09-24 12:12:00	password	USER	true	[null]
21	62	Margaret Anthony	margareta@example.org	VERIFIED	2024-10-04 08:48:00	password	USER	true	[null]
22	63	Victoria Johnson	victoriaj@gmail.com	VERIFIED	2024-08-14 16:51:00	password	USER	true	[null]
23	1	Allison Hill	allisonh@example.net	VERIFIED	2024-11-16 04:51:00	password	USER	true	[null]
24	3	Christian Santos	christians@example.net	VERIFIED	2024-10-08 01:31:00	password	USER	true	[null]
25	5	Neha Singh	nehas@example.com	VERIFIED	2024-09-15 16:09:00	password	USER	true	[null]
26	10	Rahul Sharma	rahuls@example.net	VERIFIED	2024-01-21 11:33:00	password	USER	true	[null]
27	11	Kimberly Sanchez	kimberlys@example.net	VERIFIED	2024-04-23 05:52:00	password	USER	true	[null]

Purpose: Stores the identity and account details of every person registered on the platform. It is the central entity from which orders, wallets, portfolios, and audit records all originate.

Constraints:

- PRIMARY KEY (user_id) — uniquely identifies each registered user.
- UNIQUE (email) — prevents two accounts from sharing the same email address.
- NOT NULL (name, email) — every user must have a name and email.
- CHECK chk_kyc_status: kyc_status IN ('VERIFIED','PENDING','REJECTED') — limits KYC to valid states.
- CHECK chk_role: role IN ('USER','ADMIN','TRADER','ANALYST') — prevents invalid role assignment.
- DEFAULT role = 'USER' — new registrations start with the lowest privilege level.
- DEFAULT is_active = true — accounts are active by default until explicitly deactivated

2. STOCKS

```
CREATE TABLE stocks (
  stock_id      INT      NOT NULL,
  symbol        TEXT     UNIQUE,
  company_name  TEXT     NOT NULL,
  sector        TEXT,
  is_active     BOOLEAN DEFAULT true,

  PRIMARY KEY (stock_id)
);
```

Data Output Messages Notifications					
	stock_id [PK] integer	symbol text	company_name text	sector text	is_active boolean
1	1	INFY	Infosys	IT	true
2	2	TCS	Tata Consultancy Serv...	IT	true
3	3	RELIANCE	Reliance Industries	Energy	true
4	4	HDFCBANK	HDFC Bank	Banking	true
5	5	ICICIBANK	ICICI Bank	Banking	true
6	6	LT	Larsen & Toubro	Engineering	true
7	7	ITC	ITC Limited	FMCG	true
8	8	SBIN	State Bank of India	Banking	true
9	9	WIPRO	Wipro	IT	true
10	10	AXISBANK	Axis Bank	Banking	true
11	11	MARUTI	Maruti Suzuki	Auto	true
12	12	TATAMOTO...	Tata Motors	Auto	true
13	13	ONGC	ONGC	Energy	true
14	14	NTPC	NTPC	Power	true
15	15	ADANIENT	Adani Enterprises	Conglomera...	true
16	16	ZOMATO	Zomato	Tech	true
17	17	PAYTM	One97 Communications	FinTech	true
18	18	SWIGGY	Bundl Technologies	Tech	true
19	19	BYJUS	Think & Learn	EdTech	true
20	20	NYKAA	FSN E-Commerce	Retail	true
Total rows: 20 Query complete 00:00:00.122					

Purpose: Acts as the master catalogue of all securities available for trading on the platform. Every order and portfolio entry must reference a stock listed here.

Constraints:

- PRIMARY KEY (stock_id) — uniquely identifies each listed security.
- UNIQUE (symbol) — ensures no two stocks share the same ticker (e.g. TCS, INFY).
- NOT NULL (company_name) — every stock must have a company name.
- DEFAULT is_active = true — newly listed stocks are tradeable immediately.

3.ORDERS

```
CREATE TABLE orders (  
  order_id    INT      NOT NULL DEFAULT nextval('order_seq'),  
  user_id     INT      NOT NULL,  
  stock_id    INT      NOT NULL,  
  order_type  VARCHAR,  
  order_price NUMERIC,  
  quantity    INT,  
  order_status VARCHAR,  
  created_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  
  PRIMARY KEY (order_id),  
  
  CONSTRAINT chk_order_type CHECK (UPPER(order_type) IN ('BUY', 'SELL')),  
  CONSTRAINT chk_order_price CHECK (order_price > 0),  
  CONSTRAINT chk_order_qty  CHECK (quantity > 0),  
  CONSTRAINT chk_order_status CHECK (UPPER(order_status) IN ('PENDING', 'EXECUTED', 'CANCELLED')),  
  CONSTRAINT fk_order_user  FOREIGN KEY (user_id) REFERENCES users(user_id),  
  CONSTRAINT fk_order_stock FOREIGN KEY (stock_id) REFERENCES stocks(stock_id)  
);
```

Data Output Messages Notifications									
Showing rows: 1 to 1000 Page No									
	order_id [PK] integer	user_id integer	stock_id integer	order_type character varying	order_price numeric	quantity integer	order_status character varying	created_at timestamp without time zone	
1	1	11	12	SELL	2751.10	73	PENDING	2024-11-07 17:59:02	
2	2	11	1	SELL	659.88	48	EXECUTED	2024-10-14 18:34:36	
3	3	11	1	BUY	1323.49	74	PENDING	2024-06-04 18:17:41	
4	4	11	19	BUY	2644.48	35	PENDING	2024-04-13 16:44:17	
5	5	11	20	SELL	2354.41	17	EXECUTED	2024-10-04 03:30:52	
6	6	11	13	BUY	1966.99	54	CANCELLED	2024-08-02 20:09:56	
7	7	11	12	BUY	791.11	33	PENDING	2024-08-23 07:33:22	
8	8	11	10	SELL	2745.78	45	PENDING	2024-12-13 12:05:35	
9	9	11	1	BUY	1472.21	16	PENDING	2024-03-26 07:33:00	
10	10	11	16	BUY	1599.61	28	PENDING	2024-12-30 20:37:11	
11	11	11	9	SELL	646.49	38	PENDING	2024-05-16 20:12:33	
12	12	11	6	SELL	2366.86	17	EXECUTED	2024-02-18 22:10:51	
13	13	11	8	BUY	524.46	56	EXECUTED	2024-07-24 03:49:44	
14	14	11	19	BUY	2585.20	66	EXECUTED	2024-08-13 03:12:44	
15	15	11	3	SELL	3403.17	3	PENDING	2024-12-20 13:36:50	
16	16	11	16	BUY	1022.02	70	PENDING	2024-05-15 20:52:56	
17	17	11	5	BUY	1951.89	66	EXECUTED	2024-01-16 18:31:20	
18	18	11	13	SELL	3197.32	21	PENDING	2024-12-31 09:36:14	
19	19	11	19	BUY	2032.19	12	CANCELLED	2024-07-01 06:05:42	
20	20	11	6	BUY	2188.40	75	PENDING	2024-06-02 19:29:28	
21	21	11	8	BUY	1016.53	28	PENDING	2024-02-17 06:19:24	
Total rows: 2552 Query complete 00:00:00.176									

Purpose: Records every buy or sell instruction submitted by a user. It forms the backbone of the trading engine — an order must exist before any trade can be executed.

Constraints:

- PRIMARY KEY (order_id) — uniquely identifies each order.
- FOREIGN KEY (user_id) → users(user_id) — order must belong to a registered user.
- FOREIGN KEY (stock_id) → stocks(stock_id) — order must reference a valid stock.

- NOT NULL (user_id, stock_id) — both references are mandatory.
- CHECK chk_order_type: UPPER(order_type) IN ('BUY','SELL') — only two valid order directions.
- CHECK chk_order_price: order_price > 0 — price must be a positive value.
- CHECK chk_order_qty: quantity > 0 — at least one share must be ordered.
- CHECK chk_order_status: UPPER(order_status) IN ('PENDING','EXECUTED','CANCELLED') — valid lifecycle states.
- DEFAULT created_at = CURRENT_TIMESTAMP — records submission time automatically

4. TRADES

```
CREATE TABLE trades (
  order_id      INT      NOT NULL,
  trade_id      INT      NOT NULL DEFAULT nextval('trade_seq'),
  executed_price NUMERIC,
  executed_quantity INT,
  trade_time    TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

  PRIMARY KEY (trade_id),

  CONSTRAINT chk_trade_price CHECK (executed_price > 0),
  CONSTRAINT chk_trade_qty  CHECK (executed_quantity > 0),
  CONSTRAINT fk_trade_order FOREIGN KEY (order_id) REFERENCES orders(order_id)
);
```

	order_id integer	trade_id [PK] integer	executed_price numeric	executed_quantity integer	trade_time timestamp without time zone
1	2555	3	792.03	200	2026-04-03 23:19:20.038264
2	2556	4	1920.38	100	2026-04-03 23:19:48.966112
3	2557	5	1920.38	100	2026-04-03 23:19:57.620609
4	2558	6	455.51	50	2026-04-03 23:20:27.300118
5	2559	7	175.84	200	2026-04-03 23:20:42.672768
6	2560	8	1866.30	10	2026-04-03 23:20:57.945928
7	2563	11	595.05	1	2026-04-03 23:43:31.249138
8	2564	12	595.05	1	2026-04-03 23:43:40.114872
9	2565	13	800.00	20	2026-04-04 01:33:10.861134
10	2566	14	600.00	150	2026-04-04 01:34:37.070015
11	2567	15	3620.18	1	2026-04-04 11:00:11.30326
12	2568	16	3459.58	10	2026-04-04 16:39:32.252673
13	2569	17	3459.58	5	2026-04-04 16:40:00.847049
14	2570	18	400.00	100	2026-04-04 17:36:54.635179
15	2571	19	900.62	1	2026-04-05 07:21:09.727635
16	2572	20	1500.00	5	2026-04-06 20:15:35.064087
17	2573	21	1500.00	5	2026-04-06 20:16:00.911354

Purpose: Records the actual execution of an order — the price and quantity at which shares changed hands. Separating trades from orders allows partial fills and maintains a precise execution history.

Constraints:

- PRIMARY KEY (trade_id) — uniquely identifies each execution event.
- FOREIGN KEY (order_id) → orders(order_id) — a trade can only arise from an existing order.
- NOT NULL (order_id) — every trade must trace back to an order.
- CHECK chk_trade_price: executed_price > 0 — execution price must be positive.
- CHECK chk_trade_qty: executed_quantity > 0 — at least one share must be traded.
- DEFAULT trade_time = CURRENT_TIMESTAMP — captures the exact moment of execution.

5. PORTFOLIO

```
CREATE TABLE portfolio (  
  portfolio_id INT NOT NULL DEFAULT nextval('portfolio_seq'),  
  user_id INT NOT NULL,  
  stock_id INT NOT NULL,  
  total_quantity INT,  
  avg_buy_price NUMERIC,  
  
  PRIMARY KEY (portfolio_id),  
  
  CONSTRAINT chk_port_qty CHECK (total_quantity >= 0),  
  CONSTRAINT chk_port_price CHECK (avg_buy_price >= 0),  
  CONSTRAINT fk_port_user FOREIGN KEY (user_id) REFERENCES users(user_id),  
  CONSTRAINT fk_port_stock FOREIGN KEY (stock_id) REFERENCES stocks(stock_id)  
);
```

Data Output Messages Notifications						
	portfolio_id [PK] integer	user_id integer	stock_id integer	total_quantity integer	avg_buy_price numeric	
1	1	1	6	47	1005.15	
2	2	1	18	31	677.16	
3	3	2	2	43	500.73	
4	4	2	17	29	2890.40	
5	5	2	20	49	1733.00	
6	6	3	2	32	789.35	
7	7	3	5	42	1427.80	
8	8	3	7	18	1779.67	
9	9	3	8	73	2389.58	
10	10	3	9	23	2451.50	
11	11	3	11	33	1803.72	
12	12	3	12	134	1412.57	
13	13	3	13	109	2554.33	
14	14	3	14	3	1848.94	

Purpose: Maintains the current stock holdings of each user — how many shares they own and at what average cost. This is the live ledger of a user's investment position.

Constraints:

- PRIMARY KEY (portfolio_id) — uniquely identifies each holding record.
- FOREIGN KEY (user_id) → users(user_id) — holdings belong to a valid user.
- FOREIGN KEY (stock_id) → stocks(stock_id) — holdings reference a valid stock
- NOT NULL (user_id, stock_id) — both references are mandatory.
- CHECK chk_port_qty: total_quantity ≥ 0 — share count cannot go below zero.
- CHECK chk_port_price: avg_buy_price ≥ 0 — average cost cannot be negative.

6.WALLET

```
CREATE TABLE wallet (
  wallet_id INT NOT NULL DEFAULT nextval('wallet_seq'),
  user_id INT NOT NULL,
  balance NUMERIC DEFAULT 0,
  last_updated TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

  PRIMARY KEY (wallet_id),

  CONSTRAINT chk_balance CHECK (balance >= 0),
  CONSTRAINT fk_wallet_user FOREIGN KEY (user_id) REFERENCES users(user_id)
);
```


Data Output Messages Notifications				
	wallet_id [PK] integer	user_id integer	balance numeric	last_updated timestamp without time zone
1	1	1	8210.70	2026-04-03 19:45:06.336011
2	2	2	10406.47	2024-01-10 09:22:24
3	3	3	19438.47	2024-09-28 17:00:40
4	4	4	18735.22	2024-06-19 05:34:19
5	5	5	29901.82	2024-04-02 03:25:13
6	6	6	39282.32	2024-12-13 20:57:17
7	7	7	34078.62	2024-09-28 07:22:51
8	8	8	30852.97	2024-09-24 10:18:03
9	9	9	31982.00	2024-02-29 19:08:05
10	10	10	39816.75	2024-02-13 06:29:06
11	11	11	32639.54	2024-12-21 07:33:16

Purpose: Holds the current INR balance available to each user for funding trades and receiving sale proceeds. It enforces a 1-to-1 relationship with use

Constraints:

- PRIMARY KEY (wallet_id) — uniquely identifies each wallet.
- FOREIGN KEY (user_id) → users(user_id) — wallet must belong to a registered user.
- NOT NULL (user_id) — a wallet cannot exist without an owner.
- CHECK chk_balance: balance >= 0 — wallet can never go into negative balance.
- DEFAULT balance = 0 — new wallets start with zero funds.
- DEFAULT last_updated = CURRENT_TIMESTAMP — tracks the last fund movement time.

7.WALLET_TRANSACTION

```
CREATE TABLE wallet_transactions (
    wallet_id      INT      NOT NULL,
    transaction_id INT      NOT NULL,
    amount         NUMERIC,
    transaction_type VARCHAR,
    created_at     TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

    PRIMARY KEY (wallet_id, transaction_id),

    CONSTRAINT chk_wt_amount CHECK (amount > 0),
    CONSTRAINT chk_wt_type   CHECK (UPPER(transaction_type) IN ('CREDIT', 'DEBIT')),
    CONSTRAINT fk_wt_wallet  FOREIGN KEY (wallet_id) REFERENCES wallet(wallet_id)
);
```

Data Output

Messages

Notifications

≡

+

SQL

	wallet_id [PK] integer	transaction_id [PK] integer	amount numeric	transaction_type character varying	created_at timestamp without time zone
1	1	1	40862.56	CREDIT	2024-04-30 14:58:07
2	1	2	47242.05	DEBIT	2024-04-14 20:16:58
3	1	3	112956.87	CREDIT	2024-04-14 07:37:14
4	1	4	20991.96	DEBIT	2024-01-13 08:26:51
5	1	2515	1347.37	DEBIT	2026-04-03 17:13:12.939759
6	1	2517	1347.37	DEBIT	2026-04-03 17:49:05.580574
7	1	2519	1347.37	DEBIT	2026-04-03 17:57:38.178376
8	1	2521	1347.37	DEBIT	2026-04-03 18:20:59.38274
9	1	2523	1347.37	DEBIT	2026-04-03 18:50:27.012628
10	1	2525	20598.00	DEBIT	2026-04-03 18:50:39.44015
11	1	2527	175.84	DEBIT	2026-04-03 19:37:31.233476
12	1	2529	175.84	DEBIT	2026-04-03 19:45:06.336011
13	1	2587	595.05	DEBIT	2026-04-03 23:43:31.249138
14	1	2589	595.05	CREDIT	2026-04-03 23:43:40.114872
15	1	2590	25000.00	CREDIT	2026-04-03 23:44:34.757065
16	1	2591	50000.00	CREDIT	2026-04-03 23:44:41.729515

Purpose: Provides a complete, immutable ledger of every credit (deposit, sale proceeds) and debit (order payment, withdrawal) applied to a wallet, enabling financial reconciliation and audit.

Constraints:

- PRIMARY KEY (wallet_id, transaction_id) — composite key; each transaction is unique within a wallet.
- FOREIGN KEY (wallet_id) → wallet(wallet_id) — transaction must belong to a valid wallet.
- NOT NULL (wallet_id) — transaction must always reference a wallet.

- CHECK chk_wt_amount: amount > 0 — every transaction must involve a positive monetary value.
- CHECK chk_wt_type: UPPER(transaction_type) IN ('CREDIT','DEBIT') — only two valid movement types.
- DEFAULT created_at = CURRENT_TIMESTAMP — timestamps every fund movement automatically.

8.STOCK_PRICE_HISTORY

```
CREATE TABLE stock_price_history (
  price_id      INT      NOT NULL DEFAULT nextval('price_seq'),
  stock_id      INT      NOT NULL,
  price         NUMERIC,
  price_timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

  PRIMARY KEY (price_id),

  CONSTRAINT chk_sph_price CHECK (price > 0),
  CONSTRAINT fk_sph_stock  FOREIGN KEY (stock_id) REFERENCES stocks(stock_id)
);
```

Purpose:

Stores the time-series of market prices for every stock. It supports current price lookups, historical charting, and portfolio valuation without modifying the stocks master table.

Constraints:

- PRIMARY KEY (price_id) — uniquely identifies each price record.
- FOREIGN KEY (stock_id) → stocks(stock_id) — price record must belong to a valid stock.
- NOT NULL (stock_id) — a price record must always reference a stock.
- CHECK chk_sph_price: price > 0 — market price must always be a positive number.
- DEFAULT price_timestamp = CURRENT_TIMESTAMP — records the exact time of the price update.

9.DATABASE_TRANSACTION

```
CREATE TABLE database_transactions (
    txn_id      INT      NOT NULL DEFAULT nextval('txn_seq'),
    user_id     INT,
    txn_state   VARCHAR,
    start_time  TIMESTAMP,
    end_time    TIMESTAMP,

    PRIMARY KEY (txn_id),

    CONSTRAINT chk_txn_state CHECK (
        UPPER(txn_state) IN ('ACTIVE', 'COMMITTED', 'ABORTED')
    ),
    CONSTRAINT fk_txn_user FOREIGN KEY (user_id) REFERENCES users(user_id)
);
```

Purpose: Models the DBMS-level transaction lifecycle to support concurrency control simulation. Each row represents one database transaction initiated by a user.

Constraints:

- PRIMARY KEY (txn_id) — uniquely identifies each database transaction.
- FOREIGN KEY (user_id) → users(user_id) — links the transaction to the initiating user.
- CHECK chk_txn_state: UPPER(txn_state) IN ('ACTIVE','COMMITTED','ABORTED') — enforces valid transaction states.

10.SYSTEM_LOGS

```
CREATE TABLE system_logs (
  log_id      INT          NOT NULL DEFAULT nextval('log_seq'),
  txn_id      INT,
  operation   TEXT,
  status      TEXT,
  timestamp   TIMESTAMP    DEFAULT CURRENT_TIMESTAMP,

  PRIMARY KEY (log_id),

  CONSTRAINT fk_log_txn FOREIGN KEY (txn_id) REFERENCES database_transactions(txn_id)
);
```

Data Output		Messages		Notifications	
	log_id [PK] integer	txn_id integer	operation text	status text	timestamp timestamp without time zone
1	0	[null]	BUY_ORDER FAILED: relation "txn_seq" does not e...	FAILED	2026-04-03 16:09:29.208429
2	1	2	TRADE_SUCCESS	COMMITT...	2024-10-14 18:34:37
3	2	5	TRADE_SUCCESS	COMMITT...	2024-10-04 03:30:53
4	3	12	TRADE_SUCCESS	COMMITT...	2024-02-18 22:10:52
5	4	13	TRADE_SUCCESS	COMMITT...	2024-07-24 03:49:45

11.SYSTEM_CONFIGURATION

```
CREATE TABLE system_config (  
  config_key VARCHAR NOT NULL,  
  config_value TEXT NOT NULL,  
  description TEXT,  
  data_type VARCHAR DEFAULT 'string',  
  updated_by INT,  
  updated_at TIMESTAMP DEFAULT NOW(),  
  
  PRIMARY KEY (config_key),  
  
  CONSTRAINT chk_data_type CHECK (  
    data_type IN ('string', 'number', 'boolean', 'json')  
  )  
);
```

Data Output Messages Notifications						
Showing rows: 1 to 11 Page No: 1						
	config_key [PK] character varying	config_value text	description text	data_type character varying	updated_by integer	updated_at timestamp without time zone
1	audit_retention_days	90	Days to retain audit log entries	number	[null]	2026-04-06 20:15:35.064087
2	market_close_time	15:30	Market close time (IST)	string	[null]	2026-04-06 20:15:35.064087
3	market_open_time	09:15	Market open time (IST)	string	[null]	2026-04-06 20:15:35.064087
4	max_deposit_per_txn	1000000	Maximum single deposit in INR	number	[null]	2026-04-06 20:15:35.064087
5	max_order_qty	10000	Maximum shares per single order	number	[null]	2026-04-06 20:15:35.064087
6	max_order_value	10000000	Maximum order value in INR	number	[null]	2026-04-06 20:15:35.064087
7	max_withdraw_per_txn	500000	Maximum single withdrawal in INR	number	[null]	2026-04-06 20:15:35.064087
8	min_wallet_balance	0	Minimum wallet balance allowed	number	[null]	2026-04-06 20:15:35.064087
9	platform_name	TradeFlow	Platform display name	string	[null]	2026-04-06 20:15:35.064087
10	price_update_interval	30	Stock price refresh interval in seconds	number	[null]	2026-04-06 20:15:35.064087
11	trading_enabled	true	Master switch to enable/disable trad...	boolean	[null]	2026-04-06 20:15:35.064087

Purpose: Stores platform-wide runtime parameters (market hours, transaction limits, trading on/off switch) in a key-value format, allowing administrators to tune system behaviour without code changes.

Constraints:

- PRIMARY KEY (config_key) — the key name uniquely identifies each configuration parameter.
- NOT NULL (config_value) — every parameter must have a value.
- CHECK chk_data_type: data_type IN ('string','number','boolean','json') — restricts value types to known categories.
- DEFAULT updated_at = NOW() — records when the configuration was last changed.

12.AUDIT_LOGS

```
CREATE TABLE audit_logs (  
  audit_id BIGINT NOT NULL DEFAULT nextval('audit_logs_audit_id_seq'),  
  user_id INT,  
  action VARCHAR NOT NULL,  
  resource VARCHAR,  
  resource_id TEXT,  
  details JSONB,  
  ip_address VARCHAR,  
  status VARCHAR DEFAULT 'SUCCESS',  
  created_at TIMESTAMP DEFAULT NOW(),  
  
  PRIMARY KEY (audit_id),  
  |  
  CONSTRAINT chk_audit_status CHECK (  
    status IN ('SUCCESS', 'FAILED', 'INFO')  
  ),  
  CONSTRAINT fk_audit_user FOREIGN KEY (user_id) REFERENCES users(user_id)  
);
```

Purpose: Provides an immutable record of all significant system events (order placements, logins, wallet movements, admin actions) for security monitoring, compliance, and forensic investigation.

Constraints:

- PRIMARY KEY (audit_id) — uniquely identifies each audit entry using a BIGINT to support high volume.
- FOREIGN KEY (user_id) → users(user_id) — links the event to the responsible user.
- NOT NULL (action) — every log entry must record what action occurred.
- CHECK chk_audit_status: status IN ('SUCCESS','FAILED','INFO') — restricts status to valid categories.
- DEFAULT status = 'SUCCESS' — assumes success unless the application records otherwise.
- DEFAULT created_at = NOW() — automatically timestamps every event.

13.LOCKS

```
CREATE TABLE locks (  
  lock_id      INT NOT NULL DEFAULT nextval('lock_seq'),  
  txn_id       INT,  
  resource_type TEXT,  
  lock_mode    VARCHAR,  
  
  PRIMARY KEY (lock_id),  
  
  CONSTRAINT chk_lock_mode CHECK (UPPER(lock_mode) IN ('SHARED', 'EXCLUSIVE')),  
  CONSTRAINT fk_lock_txn   FOREIGN KEY (txn_id) REFERENCES database_transactions(txn_id)  
);
```

Data Output					Messages	Notifications
	lock_id [PK] integer	txn_id integer	resource_type text	lock_mode character varying		
1	1	2	ROW	EXCLUSIVE		
2	2	5	ROW	EXCLUSIVE		
3	3	12	ROW	EXCLUSIVE		
4	4	13	ROW	EXCLUSIVE		
5	5	14	ROW	EXCLUSIVE		
6	6	17	ROW	EXCLUSIVE		
7	7	24	ROW	EXCLUSIVE		
8	8	25	ROW	EXCLUSIVE		
9	9	27	ROW	EXCLUSIVE		
10	10	28	ROW	EXCLUSIVE		
11	11	29	ROW	EXCLUSIVE		
12	12	30	ROW	EXCLUSIVE		

Purpose: Records lock acquisitions made by active transactions, enabling concurrency control analysis and deadlock detection within the system

Constraints:

- PRIMARY KEY (txn_id) — uniquely identifies each database transaction.
- FOREIGN KEY (user_id) → users(user_id) — links the transaction to the initiating user.
- CHECK chk_txn_state: UPPER(txn_state) IN ('ACTIVE','COMMITTED','ABORTED') — enforces valid transaction states.

14.ROLES

```
CREATE TABLE roles (  
  role_id      INT          NOT NULL DEFAULT nextval('roles_role_id_seq'),  
  role_name    VARCHAR(100) NOT NULL UNIQUE,  
  description  TEXT,  
  created_at   TIMESTAMP    DEFAULT NOW(),  
  
  PRIMARY KEY (role_id)  
);
```

	role_id [PK] integer	role_name character varying (100)	description text	created_at timestamp without time zone
1	1	USER	Standard trading user	2026-04-06 20:15:35.064087
2	2	ADMIN	Full system administra...	2026-04-06 20:15:35.064087
3	3	TRADER	Professional trader	2026-04-06 20:15:35.064087
4	4	ANALYST	Read-only analyst	2026-04-06 20:15:35.064087

Purpose: Defines the named roles available in the system (e.g.USER, COMPANY, ADMIN). Centralising role definitions makes it easy to add new roles without schema changes.

Constraints:

- PRIMARY KEY (role_id) — uniquely identifies each role.
- UNIQUE (role_name) — prevents duplicate role names.
- NOT NULL (role_name) — every role must have a name.
- DEFAULT created_at = NOW() — records when the role was created.

15. PERMISSIONS

```
CREATE TABLE permissions (  
  permission_id  INT          NOT NULL DEFAULT nextval('permissions_permission_id_seq'),  
  permission_name VARCHAR(100) NOT NULL UNIQUE,  
  resource       VARCHAR(100) NOT NULL,  
  action        VARCHAR(100) NOT NULL,  
  description    TEXT,  
  created_at     TIMESTAMP    DEFAULT NOW(),  
  
  PRIMARY KEY (permission_id)  
);
```

Data Output Messages Notifications						
Showing rows: 1 to 16 Page No: 1 of						
	permission_id [PK] integer	permission_name character varying (100)	resource character varying (100)	action character varying (100)	description text	created_at timestamp without time zone
1	1	users.read	users	read	View user list	2026-04-06 20:15:35.064087
2	2	users.write	users	write	Create/edit users	2026-04-06 20:15:35.064087
3	3	users.delete	users	delete	Delete users	2026-04-06 20:15:35.064087
4	4	users.role_assign	users	write	Assign roles to users	2026-04-06 20:15:35.064087
5	5	orders.read_all	orders	read	View all orders	2026-04-06 20:15:35.064087
6	6	orders.read_one	orders	read	View one order	2026-04-06 20:15:35.064087

Purpose: Defines granular resource-action pairs (e.g. orders:CREATE, audit_logs:READ) that can be granted to roles. Decoupling permissions from roles enables flexible access control.

Constraints:

- PRIMARY KEY (permission_id) — uniquely identifies each permission.
- UNIQUE (permission_name) — no two permissions can share the same name.
- NOT NULL (permission_name, resource, action) — all three fields are required for a meaningful permission.
- DEFAULT created_at = NOW() — records when the permission was defined.

16.ROLE_PERMISSIONS

```
CREATE TABLE role_permissions (  
  role_id      INT NOT NULL,  
  permission_id INT NOT NULL,  
  
  PRIMARY KEY (role_id, permission_id),  
  
  CONSTRAINT fk_rp_role      FOREIGN KEY (role_id)      REFERENCES roles(role_id),  
  CONSTRAINT fk_rp_permission FOREIGN KEY (permission_id) REFERENCES permissions(permission_id)  
);
```

Data Output Messages Notifications		
SQL		
	role_id [PK] integer	permission_id [PK] integer
1	2	1
2	2	2
3	2	3
4	2	4
5	2	5
6	2	6
7	2	7
8	2	8
9	2	9
10	2	10

Purpose: Acts as the many-to-many junction between roles and permissions, defining which actions each role is authorised to perform. This is the core of the RBAC model.

Constraints:

- PRIMARY KEY (role_id, permission_id) — composite key prevents duplicate role-permission assignments.
- FOREIGN KEY (role_id) → roles(role_id) — must reference a valid role.
- FOREIGN KEY (permission_id) → permissions(permission_id) — must reference a valid permission.
- NOT NULL (role_id, permission_id) — both sides of the mapping are mandatory

17.USER_ROLES

```
CREATE TABLE user_roles (  
  user_id      INT      NOT NULL,  
  role_id      INT      NOT NULL,  
  assigned_at  TIMESTAMP DEFAULT NOW(),  
  assigned_by  INT,  
  
  PRIMARY KEY (user_id, role_id),  
  
  CONSTRAINT fk_ur_user      FOREIGN KEY (user_id)      REFERENCES users(user_id),  
  CONSTRAINT fk_ur_role      FOREIGN KEY (role_id)        REFERENCES roles(role_id),  
  CONSTRAINT fk_ur_assigned_by FOREIGN KEY (assigned_by) REFERENCES users(user_id)  
);
```

Data Output Messages Notifications					
	user_id [PK] integer	role_id [PK] integer	assigned_at timestamp without time zone	assigned_by integer	
1	1	1	2026-04-06 20:15:35.064087	[null]	
2	2	1	2026-04-06 20:15:35.064087	[null]	
3	3	1	2026-04-06 20:15:35.064087	[null]	
4	4	1	2026-04-06 20:15:35.064087	[null]	
5	5	1	2026-04-06 20:15:35.064087	[null]	
6	6	2	2026-04-06 20:15:35.064087	[null]	

Purpose: Maps individual users to their assigned roles and records who granted the assignment. This enables auditable, multi-role user access control.

Constraints:

- PRIMARY KEY (user_id, role_id) — a user can only be assigned each role once.
- FOREIGN KEY (user_id) → users(user_id) — must reference a registered user.
- FOREIGN KEY (role_id) → roles(role_id) — must reference a defined role.
- FOREIGN KEY (assigned_by) → users(user_id) — tracks which admin performed the assignment.
- DEFAULT assigned_at = NOW() — timestamps the role assignment automatically

18. Company_profiles

```
CREATE TABLE IF NOT EXISTS public.company_profiles (  
  company_id SERIAL PRIMARY KEY,  
  user_id INT NOT NULL REFERENCES public.users(user_id) ON DELETE CASCADE,  
  stock_id INT NOT NULL REFERENCES public.stocks(stock_id) ON DELETE CASCADE,  
  company_name VARCHAR(200) NOT NULL,  
  cin_number VARCHAR(21),  
  registered_address TEXT,  
  contact_email VARCHAR(200),  
  website VARCHAR(300),  
  verified BOOLEAN DEFAULT FALSE,  
  created_at TIMESTAMP DEFAULT NOW(),  
  UNIQUE (user_id),  
  UNIQUE (stock_id)  
);
```

Purpose:

Maps companies to their associated users and stocks, while storing company details such as name, contact information, and verification status. This enables structured management of company identities linked to tradable stocks.

Constraints:

- PRIMARY KEY (company_id) — uniquely identifies each company profile.
- FOREIGN KEY (user_id) → users(user_id) — ensures the company is associated with a valid registered user.
- FOREIGN KEY (stock_id) → stocks(stock_id) — ensures the company is linked to a valid stock.
- UNIQUE (user_id) — ensures a user can have only one company profile.
- UNIQUE (stock_id) — ensures each stock is associated with only one company.
- NOT NULL (user_id, stock_id, company_name) — ensures mandatory fields are always provided.
- DEFAULT verified = FALSE — marks the company as unverified by default.
- DEFAULT created_at = NOW() — automatically records the creation timestamp.
- ON DELETE CASCADE (user_id, stock_id) — ensures related company profiles are removed if the referenced user or stock is deleted.

19. Company_announcements

```
CREATE TABLE IF NOT EXISTS public.company_announcements (  
  announcement_id SERIAL PRIMARY KEY,  
  company_id INT NOT NULL REFERENCES public.company_profiles(company_id) ON DELETE CASCADE,  
  title VARCHAR(300) NOT NULL,  
  content TEXT NOT NULL,  
  announcement_type VARCHAR(20) NOT NULL  
  CHECK (announcement_type IN ('DIVIDEND', 'SPLIT', 'BONUS', 'RESULTS', 'AGM', 'OTHER')),  
  effective_date DATE,  
  is_published BOOLEAN DEFAULT FALSE,  
  created_at TIMESTAMP DEFAULT NOW()  
);
```

Purpose:

Stores official announcements made by companies such as dividends, stock splits, results, and other corporate actions. This enables tracking of company events that may impact stock performance.

Constraints:

- PRIMARY KEY (announcement_id) — uniquely identifies each announcement.
- FOREIGN KEY (company_id) → company_profiles(company_id) — ensures announcements belong to a valid company.
- NOT NULL (title, content, announcement_type) — ensures essential announcement details are always provided.
- CHECK (announcement_type IN ('DIVIDEND', 'SPLIT', 'BONUS', 'RESULTS', 'AGM', 'OTHER')) — restricts announcement types to valid predefined values.
- DEFAULT is_published = FALSE — ensures announcements are unpublished by default.
- DEFAULT created_at = NOW() — automatically records the creation timestamp.
- ON DELETE CASCADE (company_id) — ensures announcements are deleted if the associated company is removed.

3. INTEGRITY CONSTRAINTS

3.1 PRIMARY KEY CONSTRAINTS

Every relation has a primary key that uniquely identifies each tuple. Most primary keys use auto-incrementing sequences (nextval), guaranteeing uniqueness without manual intervention. Composite primary keys are used for weak entities such as wallet_transactions (wallet_id, transaction_id) and role_permissions (role_id, permission_id).

3.2 FOREIGN KEY(REFERENTIAL INTEGRITY) CONSTRAINTS

TABLE	FOREIGN KEY COLUMN(S)	REFERENCES	DESCRIPTION
orders	user_id	users(user_id)	Every order must belong to a registered user
orders	stock_id	stocks(stock_id)	Every order must reference a valid stock
trades	order_id	orders(order_id)	A trade can only result from an existing order
portfolio	user_id	users(user_id)	Portfolio entries belong to valid users
portfolio	stock_id	stocks(stock_id)	Portfolio entries reference valid stocks
wallet	user_id	users(user_id)	Each wallet is owned by exactly one user
wallet_transactions	wallet_id	wallet(wallet_id)	Transactions recorded against valid wallets
stock_price_history	stock_id	stocks(stock_id)	Price records belong to valid stocks
role_permissions	role_id	roles(role_id)	Permission mappings reference valid roles
role_permissions	permission_id	permissions(permission_id)	Permission mappings reference valid permissions
user_roles	user_id	users(user_id)	Role assignments reference valid users
user_roles	assigned_by	roles(role_id)	Role assignments reference valid roles
user_roles	role_id	users(user_id)	Tracks which admin assigned the role
audit_logs	user_id	users(user_id)	Audit entries linked to valid users
database_transactions	user_id	users(user_id)	DB transactions belong to valid users
locks	txn_id	database_transactions(txn_id)	Locks are acquired within valid transactions
system_logs	txn_id	database_transac	System logs tied to valid DB transactions

		tions(txn_id)	
--	--	---------------	--

3.3 CHECK CONSTRAINTS

CONSTRAINTS NAME	TABLE	RULE	PURPOSE
chk_kyc_status	users	kyc_status IN ('VERIFIED','PENDING','REJECTED')	Limits KYC to valid states
chk_role	users	role IN ('USER','ADMIN','TRADER','ANALYST')	Prevents invalid role assignment
chk_balance	wallet	balance >= 0	Wallet cannot go negative
chk_order_type	orders	UPPER(order_type) IN ('BUY','SELL')	Only buy/sell orders allowed
chk_order_price	orders	order_price > 0	Price must be positive
chk_order_qty	orders	quantity > 0	Quantity must be at least 1
chk_order_status	orders	UPPER(order_status) IN ('PENDING','EXECUTED','CANCELLED')	Valid order lifecycle states
chk_trade_price	trades	executed_price > 0	Trade price must be positive
chk_trade_qty	trades	executed_quantity > 0	Trade quantity must be positive
chk_port_qty	portfolio	total_quantity >= 0	Holdings cannot be negative
chk_port_price	portfolio	avg_buy_price >= 0	Avg price cannot be negative
chk_wt_amount	wallet_transactions	amount > 0	Transaction amount must be positive
chk_wt_type	wallet_transactions	UPPER(transaction_type) IN ('CREDIT','DEBIT')	Only credit/debit allowed
chk_sph_price	stock_price_history	price > 0	Stock price must be positive

chk_txn_state	database_transactions	UPPER(txn_state) IN ('ACTIVE','COMMITTED','ABORTED')	Valid transaction states
chk_lock_mode	locks	UPPER(lock_mode) IN ('SHARED','EXCLUSIVE')	Only two valid lock modes
chk_audit_status	audit_logs	status IN ('SUCCESS','FAILED','INFO')	Valid audit statuses
chk_data_type	system_config	data_type IN ('string','number','boolean','json')	Valid config value types

3.4 NOT NULL CONSTRAINTS

Critical fields that must always carry a value include: users.name, users.email, orders.user_id, orders.stock_id, stocks.company_name, wallet.user_id, wallet_transactions.wallet_id, audit_logs.action, permissions.resource, permissions.action, roles.role_name, and system_config.config_value. These constraints prevent incomplete or orphaned records.

3.5 UNIQUE CONSTRAINTS

Uniqueness is enforced on users.email (no duplicate accounts), stocks.symbol (unique ticker, e.g. TCS, INFY), roles.role_name, and permissions.permission_name. These prevent logical duplicates that primary keys alone

3.6 DEFAULT CONSTRAINTS

Sensible defaults reduce the burden on application code: wallet.balance defaults to 0, users.is_active defaults to true, users.role defaults to 'USER', stocks.is_active defaults to true, audit_logs.status defaults to 'SUCCESS', and all timestamp columns default to CURRENT_TIMESTAMP / NOW().

4. FUNCTIONS and TRIGGERS

The system implements 21 stored functions written in PL/pgSQL, organised into five groups: Trading Core, Wallet Management, Portfolio & Analytics, Auth & Audit, and Concurrency Control. Together they enforce business rules atomically at the database layer, independently of the calling application.

4.1 FUNCTIONS

1. CREDIT WALLET

```
-- ---- 6b. CreditWallet -----
-- Adds funds to a wallet and records a CREDIT transaction.
-- Called on: deposit, sell-order settlement.

CREATE FUNCTION CreditWallet(p_user_id INTEGER, p_amount NUMERIC)
RETURNS VOID
LANGUAGE plpgsql
AS $$
DECLARE
    v_wallet_id INTEGER;
BEGIN
    IF p_amount <= 0 THEN
        RAISE EXCEPTION 'INVALID_AMOUNT: must be greater than 0, got %', p_amount;
    END IF;

    SELECT wallet_id INTO v_wallet_id
    FROM wallet
    WHERE user_id = p_user_id
    FOR UPDATE;

    IF NOT FOUND THEN
        RAISE EXCEPTION 'WALLET_NOT_FOUND: no wallet for user_id %', p_user_id;
    END IF;

    UPDATE wallet
    SET balance = balance + p_amount,
        last_updated = NOW()
    WHERE wallet_id = v_wallet_id;

    INSERT INTO wallet_transactions(transaction_id, wallet_id, amount, transaction_type, created_at)
    VALUES (nextval('txn_seq'), v_wallet_id, p_amount, 'CREDIT', NOW());
END;
$$;
```

Function_name: CreditWallet(p_user_id INT, p_amount NUMERIC) → VOID

What it does: Validates amount > 0, acquires a FOR UPDATE lock on the wallet row, adds p_amount to balance, updates last_updated = NOW(), and inserts a CREDIT row into wallet_transactions.

How it preserves consistency: The FOR UPDATE lock serialises concurrent credits. Every credit is traceable through wallet_transactions, enabling financial reconciliation.

2. DEBIT WALLET

```
-- ----- Gc. DebitWallet -----
-- Removes funds from a wallet and records a DEBIT transaction.
-- Called on: withdrawal, buy-order settlement.

CREATE FUNCTION DebitWallet(p_user_id INTEGER, p_amount NUMERIC)
RETURNS VOID
LANGUAGE plpgsql
AS $$
DECLARE
    v_wallet_id INTEGER;
    v_balance    NUMERIC;
BEGIN
    IF p_amount <= 0 THEN
        RAISE EXCEPTION 'INVALID_AMOUNT: must be greater than 0, got %', p_amount;
    END IF;
    SELECT wallet_id, balance INTO v_wallet_id, v_balance
    FROM wallet
    WHERE user_id = p_user_id
    FOR UPDATE;
    IF NOT FOUND THEN
        RAISE EXCEPTION 'WALLET_NOT_FOUND: no wallet for user_id %', p_user_id;
    END IF;
    IF v_balance < p_amount THEN
        RAISE EXCEPTION 'INSUFFICIENT_FUNDS: balance ₹% < required ₹%', v_balance, p_amount;
    END IF;
    UPDATE wallet
    SET balance      = balance - p_amount,
        last_updated = NOW()
    WHERE wallet_id = v_wallet_id;
    INSERT INTO wallet_transactions(transaction_id, wallet_id, amount, transaction_type, created_at)
    VALUES (nextval('txn_seq'), v_wallet_id, p_amount, 'DEBIT', NOW());
END;
$$;
```

FUNCTION_NAME: DebitWallet(p_user_id INT, p_amount NUMERIC) → VOID

What it does: Validates amount > 0, locks the wallet row (FOR UPDATE), checks that balance >= p_amount (raises INSUFFICIENT_FUNDS otherwise), deducts p_amount, updates last_updated, and inserts a DEBIT row into wallet_transactions.

How it preserves consistency: The explicit balance check before deduction is the primary guard against negative balances, complementing the CHECK constraint on wallet.balance. The FOR UPDATE lock prevents race conditions where two concurrent debits could both pass the balance check.

3. UPDATE WALLET

```
-- ---- 6d. UpdateWallet -----  
-- Thin dispatcher: calls CreditWallet for SELL, DebitWallet for BUY.  
  
CREATE FUNCTION UpdateWallet(  
    p_user_id    INTEGER,  
    p_amount     NUMERIC,  
    p_order_type VARCHAR  
)  
RETURNS VOID  
LANGUAGE plpgsql  
AS $$  
BEGIN  
    IF p_order_type = 'BUY' THEN  
        PERFORM DebitWallet(p_user_id, p_amount);  
    ELSE  
        PERFORM CreditWallet(p_user_id, p_amount);  
    END IF;  
END;  
$$;
```

FUNCTION_NAME: UpdateWallet(p_user_id INT, p_amount NUMERIC, p_order_type VARCHAR) → VOID

What it does: Thin dispatcher: calls DebitWallet for BUY orders and CreditWallet for SELL orders. Called by ExecuteOrder to settle the wallet after a trade.

How it preserves consistency: Centralises the wallet-settlement decision in one place, ensuring the correct direction (debit vs credit) is always applied consistently for every trade type.

4. UPDATE PORTFOLIO

```
-- ---- 6e. UpdatePortfolio -----
-- Upserts the portfolio holding for a given user + stock.
--
-- BUY: creates a new row or increments quantity with a new weighted average.
-- SELL: decrements quantity. Sets to 0 if selling all shares (row kept for
--       the portfolio_transactions FK — a separate cleanup job can prune zeroes).

CREATE FUNCTION UpdatePortfolio(
    p_user_id    INTEGER,
    p_stock_id   INTEGER,
    p_quantity   INTEGER,
    p_price      NUMERIC,
    p_order_type VARCHAR
)
RETURNS VOID
LANGUAGE plpgsql
AS $$
DECLARE
    v_portfolio_id INTEGER;
    v_existing_qty  INTEGER;
    v_existing_price NUMERIC;
    v_new_avg      NUMERIC;
BEGIN
    SELECT portfolio_id, total_quantity, avg_buy_price
    INTO   v_portfolio_id, v_existing_qty, v_existing_price
    FROM   portfolio
    WHERE  user_id = p_user_id
    AND    stock_id = p_stock_id
    FOR UPDATE;

    IF p_order_type = 'BUY' THEN
        IF FOUND THEN
            -- Weighted average: ((old_qty * old_avg) + (new_qty * new_price)) / total_qty
            v_new_avg := (
                (v_existing_qty * v_existing_price) + (p_quantity * p_price)
            ) / (v_existing_qty + p_quantity);

            UPDATE portfolio
            SET total_quantity = total_quantity + p_quantity,
                avg_buy_price  = ROUND(v_new_avg, 2)
            WHERE user_id = p_user_id AND stock_id = p_stock_id;
        ELSE
            INSERT INTO portfolio(portfolio_id, user_id, stock_id, total_quantity, avg_buy_price)
            VALUES (nextval('portfolio_seq'), p_user_id, p_stock_id, p_quantity, p_price);
        END IF;
    ELSEIF p_order_type = 'SELL' THEN
        IF NOT FOUND OR v_existing_qty < p_quantity THEN
            RAISE EXCEPTION 'INSUFFICIENT_HOLDINGS: cannot sell % shares, only % held',
                p_quantity, COALESCE(v_existing_qty, 0);
        END IF;

        -- We update to the new quantity (could reach 0).
        -- We never DELETE the row here so that portfolio_transactions keeps its FK.
        UPDATE portfolio
        SET total_quantity = total_quantity - p_quantity
        WHERE user_id = p_user_id AND stock_id = p_stock_id;
    END IF;
END;
$$;
```

Signature: UpdatePortfolio(p_user_id INT, p_stock_id INT, p_quantity INT, p_price NUMERIC, p_order_type VARCHAR) → VOID

What it does: Upserts the portfolio holding. For BUY: if a row exists, increments total_quantity and recalculates avg_buy_price using a weighted

average; if not, inserts a new row. For SELL: verifies sufficient holdings, then decrements total_quantity (sets to 0 if all shares sold — row is kept to preserve portfolio_transactions FK).

How it preserves consistency: The weighted-average formula keeps avg_buy_price accurate across multiple purchases. The INSUFFICIENT_HOLDINGS guard prevents selling shares the user does not own.

5. CREATE TABLE

```
-- ---- 6f. CreateTrade ----  
-- Inserts the trade record and the corresponding portfolio_transactions entry.  
-- Must be called AFTER UpdatePortfolio so the portfolio row already exists.  
  
CREATE FUNCTION CreateTrade(  
    p_order_id INTEGER,  
    p_user_id  INTEGER,  
    p_stock_id INTEGER,  
    p_price    NUMERIC,  
    p_quantity INTEGER  
)  
RETURNS INTEGER  
LANGUAGE plpgsql  
AS $$  
DECLARE  
    v_trade_id    INTEGER;  
    v_portfolio_id INTEGER;  
    v_order_type  VARCHAR;  
BEGIN  
    SELECT order_type INTO v_order_type  
    FROM orders  
    WHERE order_id = p_order_id;  
  
    IF NOT FOUND THEN  
        RAISE EXCEPTION 'ORDER_NOT_FOUND: order_id % does not exist', p_order_id;  
    END IF;  
  
    v_trade_id := nextval('trade_seq');  
  
    INSERT INTO trades(trade_id, order_id, executed_price, executed_quantity, trade_time)  
    VALUES (v_trade_id, p_order_id, p_price, p_quantity, NOW());  
  
    -- Portfolio row exists (or is zero) at this point – fetch its ID for the ledger.  
    SELECT portfolio_id INTO v_portfolio_id  
    FROM portfolio  
    WHERE user_id = p_user_id AND stock_id = p_stock_id;  
  
    INSERT INTO portfolio_transactions(portfolio_id, order_id, trade_id, quantity_change, created_at)  
    VALUES (  
        v_portfolio_id,  
        p_order_id,  
        v_trade_id,  
        CASE WHEN v_order_type = 'BUY' THEN p_quantity ELSE -p_quantity END,  
        NOW()  
    );  
  
    RETURN v_trade_id;  
END;  
$$;
```

FUNCTION_NAME: CreateTrade(p_order_id INT, p_user_id INT, p_stock_id INT, p_price NUMERIC, p_quantity INT) → INTEGER

What it does: Inserts a row into trades (executed_price, executed_quantity, trade_time=NOW()) and a corresponding row into portfolio_transactions capturing the quantity_change (positive for BUY, negative for SELL). Must be called after UpdatePortfolio so the portfolio row already exists.

How it preserves consistency: Separating trade recording from order execution ensures the trades table is always consistent with

portfolio_transactions, and both are written in the same atomic call chain as ExecuteOrder

6. PLACE ORDER

```
-- ---- 6g. PlaceOrder ----  
-- Validates inputs and inserts a PENDING order. Returns the new order_id.  
-- Does NOT touch the wallet or portfolio - that happens in ExecuteOrder().  
  
CREATE FUNCTION PlaceOrder(  
    p_user_id    INTEGER,  
    p_stock_id   INTEGER,  
    p_order_type VARCHAR,  
    p_quantity   INTEGER,  
    p_price      NUMERIC  
)  
RETURNS INTEGER  
LANGUAGE plpgsql  
AS $$  
DECLARE  
    v_order_id INTEGER;  
BEGIN  
    IF NOT EXISTS (SELECT 1 FROM users WHERE user_id = p_user_id AND is_active = TRUE) THEN  
        RAISE EXCEPTION 'USER_NOT_FOUND: user_id % does not exist or is inactive', p_user_id;  
    END IF;  
  
    IF NOT EXISTS (SELECT 1 FROM stocks WHERE stock_id = p_stock_id AND is_active = TRUE) THEN  
        RAISE EXCEPTION 'STOCK_NOT_FOUND: stock_id % not found or inactive', p_stock_id;  
    END IF;  
  
    IF p_order_type NOT IN ('BUY', 'SELL') THEN  
        RAISE EXCEPTION 'INVALID_ORDER_TYPE: must be BUY or SELL, got %', p_order_type;  
    END IF;  
  
    IF p_quantity <= 0 OR p_price <= 0 THEN  
        RAISE EXCEPTION 'INVALID_PARAMS: quantity and price must both be greater than 0';  
    END IF;  
  
    v_order_id := nextval('order_seq');  
  
    INSERT INTO orders(order_id, user_id, stock_id, order_type, order_price,  
        quantity, order_status, created_at)  
    VALUES (v_order_id, p_user_id, p_stock_id, p_order_type, p_price,  
        p_quantity, 'PENDING', NOW());  
  
    PERFORM LogOperation(NULL, 'ORDER_PLACED order_id=' || v_order_id, 'SUCCESS');  
  
    RETURN v_order_id;  
END;  
$$;
```

FUNCTION_NAME: PlaceOrder(p_user_id INT, p_stock_id INT, p_order_type VARCHAR, p_quantity INT, p_price NUMERIC) → INTEGER

What it does: Validates that the user is active, the stock is active, the order type is BUY/SELL, and that quantity and price are positive. Inserts a PENDING order and logs the event. Returns the new order_id.

How it preserves consistency: All four guard conditions raise exception before any insert, so an invalid order can never reach the order table. Calling LogOperation ensures every placement is recorded in system_logs.

7. . EXECUTE ORDER

```
-- ---- 6h. ExecuteOrder -----
-- The main trading entry point. Steps in order:
-- 1. Lock the order row and validate it is still PENDING.
-- 2. Open a database_transactions record.
-- 3. Mark order EXECUTED.
-- 4. Settle the wallet (debit for BUY, credit for SELL).
-- 5. Update the portfolio holding.
-- 6. Create the trade and portfolio_transactions records.
-- 7. Commit the database_transactions record and write a log entry.
-- Returns the trade_id on success.

CREATE FUNCTION ExecuteOrder(p_order_id INTEGER)
RETURNS INTEGER
LANGUAGE plpgsql
AS $$
DECLARE
    v_order      orders%ROWTYPE;
    v_trade_id   INTEGER;
    v_amount     NUMERIC;
    v_txn_id     INTEGER;
BEGIN
    SELECT * INTO v_order
    FROM orders
    WHERE order_id = p_order_id
    FOR UPDATE;

    IF NOT FOUND THEN
        RAISE EXCEPTION 'ORDER_NOT_FOUND: order_id % does not exist', p_order_id;
    END IF;

    IF v_order.order_status != 'PENDING' THEN
        RAISE EXCEPTION 'ORDER_NOT_PENDING: order % has status %, cannot execute',
            p_order_id, v_order.order_status;
    END IF;

    -- Open a transaction record so we can track commit/abort
    v_txn_id := nextval('txn_seq');
    INSERT INTO database_transactions(txn_id, user_id, txn_state, start_time)
    VALUES (v_txn_id, v_order.user_id, 'ACTIVE', NOW());

    -- Mark the order executed
    UPDATE orders SET order_status = 'EXECUTED' WHERE order_id = p_order_id;

    v_amount := v_order.order_price * v_order.quantity;

    -- Commit the transaction record
    UPDATE database_transactions
    SET txn_state = 'COMMITTED', end_time = NOW()
    WHERE txn_id = v_txn_id;

    PERFORM LogOperation(
        v_txn_id,
        v_order.order_type || '_EXECUTED order_id=' || p_order_id
        || ' qty=' || v_order.quantity
        || ' price=' || v_order.order_price,
        'SUCCESS'
    );

    RETURN v_trade_id;

EXCEPTION WHEN OTHERS THEN
    -- Roll back the transaction record if anything went wrong
    UPDATE database_transactions
    SET txn_state = 'ABORTED', end_time = NOW()
    WHERE txn_id = v_txn_id;

    RAISE;
END;
$$;
```

FUNCTION_NAME: ExecuteOrder(p_order_id INT) → INTEGER

What it does: The main trading entry point. Locks the order row (FOR UPDATE), opens a database_transactions record, marks the order EXECUTED, settles the wallet (DebitWallet for BUY, CreditWallet for SELL), calls UpdatePortfolio and CreateTrade, then commits the transaction record. On any exception, the transaction is marked ABORTED. Returns trade_id.

How it preserves consistency: The FOR UPDATE lock prevents two concurrent calls from executing the same order twice. Wrapping all steps in a single PL/pgSQL block means the order, trade, portfolio, and wallet are updated atomically — a failure at any step aborts the whole operation.

8. CANCEL ORDER

```
-- ---- 6i. CancelOrder -----
-- Moves a PENDING order to CANCELLED. Cannot cancel already-executed orders.

CREATE FUNCTION CancelOrder(p_order_id INTEGER)
RETURNS INTEGER
LANGUAGE plpgsql
AS $$
DECLARE
    v_order orders%ROWTYPE;
BEGIN
    SELECT * INTO v_order
    FROM orders
    WHERE order_id = p_order_id
    FOR UPDATE;

    IF NOT FOUND THEN
        RAISE EXCEPTION 'ORDER_NOT_FOUND: order_id % does not exist', p_order_id;
    END IF;

    IF v_order.order_status != 'PENDING' THEN
        RAISE EXCEPTION 'ORDER_NOT_PENDING: order % has status %, cannot cancel',
            p_order_id, v_order.order_status;
    END IF;

    UPDATE orders SET order_status = 'CANCELLED' WHERE order_id = p_order_id;

    PERFORM LogOperation(NULL, 'ORDER_CANCELLED order_id=' || p_order_id, 'SUCCESS');

    RETURN p_order_id;
END;
$$;
```

FUNCTION_NAME: CancelOrder(p_order_id INT) → INTEGER

What it does: Locks the order row, verifies it is still PENDING (raises exception otherwise), sets order_status = 'CANCELLED', and logs the cancellation.

How it preserves consistency: The PENDING check prevents cancellation of already-executed or already-cancelled orders, keeping the order lifecycle strictly one-directional.

9. AcquireLock

```
-- ---- 6j. AcquireLock / ReleaseLock -----  
-- Thin helpers used to demonstrate shared vs. exclusive locking in the report.  
  
CREATE FUNCTION AcquireLock(  
    p_txn_id    INTEGER,  
    p_resource  TEXT,  
    p_mode      VARCHAR DEFAULT 'EXCLUSIVE'  
)  
RETURNS INTEGER  
LANGUAGE plpgsql  
AS $$  
DECLARE  
    v_lock_id INTEGER;  
BEGIN  
    v_lock_id := nextval('lock_seq');  
  
    INSERT INTO locks(lock_id, txn_id, resource_type, lock_mode)  
    VALUES (v_lock_id, p_txn_id, p_resource, p_mode);  
  
    RETURN v_lock_id;  
END;  
$$;  
  
CREATE FUNCTION ReleaseLock(p_lock_id INTEGER)  
RETURNS VOID  
LANGUAGE plpgsql  
AS $$  
BEGIN  
    DELETE FROM locks WHERE lock_id = p_lock_id;  
END;  
$$;
```

Function_name: AcquireLock(p_txn_id INT, p_resource TEXT, p_mode VARCHAR DEFAULT 'EXCLUSIVE') → INTEGER

What it does: Records a lock acquisition in the locks table for the given transaction, resource, and mode (SHARED or EXCLUSIVE). Returns the new lock_id.

How it preserves consistency: Provides an application-level lock log for concurrency analysis and deadlock detection, complementing PostgreSQL's internal locking.

10. GetPortfolioValue

```
-- ---- 7a. GetPortfolioValue -----  
-- Returns the total current market value of all holdings for a user.  
|  
CREATE FUNCTION GetPortfolioValue(p_user_id INTEGER)  
RETURNS NUMERIC  
LANGUAGE plpgsql  
AS $$  
BEGIN  
    RETURN (  
        SELECT COALESCE(  
            SUM(  
                p.total_quantity * COALESCE(  
                    (SELECT sph.price  
                     FROM stock_price_history sph  
                     WHERE sph.stock_id = p.stock_id  
                     ORDER BY sph.price_timestamp DESC  
                     LIMIT 1),  
                    p.avg_buy_price  
                )  
            ),  
            0)  
        FROM portfolio p  
        WHERE p.user_id = p_user_id  
            AND p.total_quantity > 0  
    );  
END;  
$$;
```

FUNCTION_NAME: GetPortfolioValue(p_user_id INT) → NUMERIC

What it does: Returns the total current market value of all active holdings (total_quantity > 0) by multiplying each holding's total_quantity by the most recent price from stock_price_history. Falls back to avg_buy_price if no price history exists.

How it preserves consistency: Calculates live portfolio value without storing derived data, keeping the schema normalised and preventing stale valuations.

11. GetProfitLoss

```

-- ---- 7b. GetProfitLoss -----
-- Returns total unrealised P&L (current value minus cost basis).

CREATE FUNCTION GetProfitLoss(p_user_id INTEGER)
RETURNS NUMERIC
LANGUAGE plpgsql
AS $$
BEGIN
    RETURN (
        SELECT COALESCE(
            SUM(
                p.total_quantity * (
                    COALESCE(
                        (SELECT sph.price
                         FROM stock_price_history sph
                         WHERE sph.stock_id = p.stock_id
                         ORDER BY sph.price_timestamp DESC
                         LIMIT 1),
                        p.avg_buy_price
                    ) - p.avg_buy_price
            ),
            0)
        FROM portfolio p
        WHERE p.user_id = p_user_id
        AND p.total_quantity > 0
    );
END;
$$;

```

FUNCTION_NAME: GetProfitLoss(p_user_id INT) → NUMERIC

What it does: Returns the total unrealised P&L; for a user: SUM of (latest_price - avg_buy_price) * total_quantity across all holdings with quantity > 0.

How it preserves consistency: Surfaces P&L; as a single number without storing derived data, allowing the dashboard to show real-time gain/loss without schema denormalisation.

12. GetWalletBalance

```

-- ---- 7c. GetWalletBalance -----
-- Returns just the current balance for a user. Useful for quick checks.

CREATE FUNCTION GetWalletBalance(p_user_id INTEGER)
RETURNS NUMERIC
LANGUAGE plpgsql
AS $$
DECLARE
    v_balance NUMERIC;
BEGIN
    SELECT balance INTO v_balance FROM wallet WHERE user_id = p_user_id;
    RETURN COALESCE(v_balance, 0);
END;
$$;

```

Function call: GetWalletBalance(p_user_id INT) → NUMERIC

What it does: Returns the current balance for a user's wallet using COALESCE to return 0 if no wallet exists.

How it preserves consistency: Provides a safe, null-free balance value that calling code can use directly for pre-order fund checks without risk of null-comparison errors.

13. GetWalletSummary

```
-- -- 7d. GetWalletSummary -----
-- Returns balance and running credit/debit totals for a user's wallet.

CREATE FUNCTION GetWalletSummary(p_user_id INTEGER)
RETURNS TABLE(
    wallet_id      INTEGER,
    balance        NUMERIC,
    total_credited NUMERIC,
    total_debited  NUMERIC,
    txn_count      BIGINT,
    last_updated   TIMESTAMP
)
LANGUAGE plpgsql
AS $$
BEGIN
    RETURN QUERY
    SELECT
        w.wallet_id,
        w.balance,
        COALESCE(SUM(CASE WHEN wt.transaction_type = 'CREDIT' THEN wt.amount ELSE 0 END), 0),
        COALESCE(SUM(CASE WHEN wt.transaction_type = 'DEBIT' THEN wt.amount ELSE 0 END), 0),
        COUNT(wt.transaction_id),
        w.last_updated
    FROM wallet w
    LEFT JOIN wallet_transactions wt ON wt.wallet_id = w.wallet_id
    WHERE w.user_id = p_user_id
    GROUP BY w.wallet_id, w.balance, w.last_updated;
END;
$$;
```

Signature: GetWalletSummary(p_user_id INT) → TABLE(wallet_id, total_credited, total_debited, txn_count, last_updated) balance,

What it does: Joins wallet with wallet_transactions to return the current balance alongside aggregated CREDIT total, DEBIT total, transaction count, and last_updated timestamp.

How it preserves consistency: Exposes a full financial summary in a single call, supporting account statements and reconciliation checks without requiring the caller to join multiple tables.

14. GetUserDashboard

```
-- ---- 7e. GetUserDashboard -----
-- Single-call summary for the user dashboard: portfolio value, wallet,
-- P&L, and order counts.

CREATE FUNCTION GetUserDashboard(p_user_id INTEGER)
RETURNS TABLE(
    portfolio_value NUMERIC,
    wallet_balance  NUMERIC,
    unrealized_pnl  NUMERIC,
    total_invested  NUMERIC,
    open_orders     BIGINT,
    total_trades     BIGINT
)
LANGUAGE plpgsql
AS $$
BEGIN
    RETURN QUERY
    SELECT
        GetPortfolioValue(p_user_id),
        GetWalletBalance(p_user_id),
        GetProfitLoss(p_user_id),
        COALESCE(
            (SELECT SUM(total_quantity * avg_buy_price)
             FROM portfolio
             WHERE user_id = p_user_id AND total_quantity > 0),
            0),
        (SELECT COUNT(*) FROM orders
         WHERE user_id = p_user_id AND order_status = 'PENDING')::BIGINT,
        (SELECT COUNT(*) FROM trades t
         JOIN orders o ON o.order_id = t.order_id
         WHERE o.user_id = p_user_id)::BIGINT;
END;
$$;
```

Signature: GetUserDashboard(p_user_id INT) → TABLE(portfolio_value, wallet_balance, unrealized_pnl, total_invested, open_orders, total_trades)

What it does: Single-call dashboard aggregation: calls GetPortfolioValue, GetWalletBalance, GetProfitLoss internally and adds total invested (cost basis), count of PENDING orders, and total executed trades.

How it preserves consistency: Composing the three sub-functions means each calculation is reused consistently. Returning everything in one call reduces round-trips and prevents the dashboard showing inconsistent snapshots across multiple queries.

15. GetUserOrderSummary

```
-- ---- 7f. GetUserOrderSummary -----  
-- Order counts and buy/sell volume totals for a user.  
  
CREATE FUNCTION GetUserOrderSummary(p_user_id INTEGER)  
RETURNS TABLE(  
    total_orders      BIGINT,  
    executed_orders   BIGINT,  
    pending_orders    BIGINT,  
    cancelled_orders  BIGINT,  
    total_buy_value   NUMERIC,  
    total_sell_value  NUMERIC  
)  
LANGUAGE plpgsql  
AS $$  
BEGIN  
    RETURN QUERY  
    SELECT  
        COUNT(*),  
        COUNT(*) FILTER (WHERE order_status = 'EXECUTED'),  
        COUNT(*) FILTER (WHERE order_status = 'PENDING'),  
        COUNT(*) FILTER (WHERE order_status = 'CANCELLED'),  
        COALESCE(SUM(CASE WHEN order_type = 'BUY' THEN order_price * quantity ELSE 0 END), 0),  
        COALESCE(SUM(CASE WHEN order_type = 'SELL' THEN order_price * quantity ELSE 0 END), 0)  
    FROM orders  
    WHERE user_id = p_user_id;  
END;  
$$;
```

Function_name: GetUserOrderSummary(p_user_id INT) →
TABLE(total_orders, executed_orders, pending_orders, cancelled_orders,
total_buy_value, total_sell_value)

What it does: Returns order-count breakdown and total BUY/SELL value for a user from the orders table in a single query.

How it preserves consistency: Provides trading activity metrics in one call, supporting account statements and analytics without requiring the application to issue multiple COUNT queries.

16. GetAdminStats

```

CREATE FUNCTION GetAdminStats()
RETURNS TABLE(
    total_users          BIGINT,
    active_users         BIGINT,
    total_orders         BIGINT,
    executed_orders      BIGINT,
    pending_orders       BIGINT,
    total_trades         BIGINT,
    total_volume         NUMERIC,
    total_wallet_bal     NUMERIC,
    active_stocks        BIGINT,
    audit_events_24h     BIGINT
)
LANGUAGE plpgsql
AS $$
BEGIN
    RETURN QUERY SELECT
        (SELECT COUNT(*) FROM users)::BIGINT,
        (SELECT COUNT(*) FROM users WHERE is_active = TRUE)::BIGINT,
        (SELECT COUNT(*) FROM orders)::BIGINT,
        (SELECT COUNT(*) FROM orders WHERE order_status = 'EXECUTED')::BIGINT,
        (SELECT COUNT(*) FROM orders WHERE order_status = 'PENDING')::BIGINT,
        (SELECT COUNT(*) FROM trades)::BIGINT,
        (SELECT COALESCE(SUM(executed_price * executed_quantity), 0) FROM trades),
        (SELECT COALESCE(SUM(balance), 0) FROM wallet),
        (SELECT COUNT(*) FROM stocks WHERE is_active = TRUE)::BIGINT,
        (SELECT COUNT(*) FROM audit_logs
         WHERE created_at >= NOW() - INTERVAL '24 hours')::BIGINT;
END;
$$;

```

Function_name: GetAdminStats() → TABLE(total_users, active_users, total_orders, executed_orders, pending_orders, total_trades, total_volume, total_wallet_bal, active_stocks, audit_events_24h)

What it does: Platform-wide dashboard for admins: ten metrics collected in a single query using correlated subselects — user counts, order counts, trade volume, total wallet balance, active stocks, and audit events in the last 24 hours.

How it preserves consistency: Aggregating all admin metrics server-side in one function prevents the admin dashboard from issuing ten separate queries and eliminates the risk of reading inconsistent snapshots.

17. GetAdminUserProfile

Function Name: GetAdminUserProfile(p_user_id INT) → TABLE(user_id, name, email, role, kyc_status, is_active, last_login, created_at, wallet_balance, total_orders, total_trades, portfolio_value)

What it does: Returns a full admin-level profile for a user: all identity fields plus wallet balance, order count, trade count, and live portfolio value — all in one query using LEFT JOINS and sub-calls to GetPortfolioValue.

How it preserves consistency: Combining user, wallet, and activity data server-side means the admin view is always consistent and does not require the application to join sensitive tables directly.

```
CREATE FUNCTION GetAdminUserProfile(p_user_id INTEGER)
RETURNS TABLE(
    user_id      INTEGER,
    name         VARCHAR,
    email        VARCHAR,
    role         VARCHAR,
    kyc_status   VARCHAR,
    is_active    BOOLEAN,
    last_login   TIMESTAMP,
    created_at   TIMESTAMP,
    wallet_balance NUMERIC,
    total_orders BIGINT,
    total_trades BIGINT,
    portfolio_value NUMERIC
)
LANGUAGE plpgsql
AS $$
BEGIN
    RETURN QUERY
    SELECT
        u.user_id,
        u.name,
        u.email,
        u.role,
        u.kyc_status,
        u.is_active,
        u.last_login,
        u.created_at,
        COALESCE(w.balance, 0),
        (SELECT COUNT(*) FROM orders o WHERE o.user_id = u.user_id)::BIGINT,
        (SELECT COUNT(*) FROM trades t
         JOIN orders o2 ON o2.order_id = t.order_id
         WHERE o2.user_id = u.user_id)::BIGINT,
        GetPortfolioValue(u.user_id)
    FROM users u
    LEFT JOIN wallet w ON w.user_id = u.user_id
    WHERE u.user_id = p_user_id;
END;
$$;
```

18. WriteAuditLog

Function_name WriteAuditLog(p_user_id INT, p_action TEXT, p_resource TEXT, p_res_id TEXT, p_details TEXT, p_status TEXT, p_ip TEXT) → BIGINT

What it does: Inserts a row into audit_logs, casting p_details to JSONB if provided. Returns the new audit_id. Used by other functions and the application layer.

How it preserves consistency: Centralising audit writes in one function guarantees that every caller uses the same column mapping and JSONB casting, preventing malformed audit records.

```
CREATE FUNCTION WriteAuditLog(  
  p_user_id INTEGER,  
  p_action TEXT,  
  p_resource TEXT DEFAULT NULL,  
  p_res_id TEXT DEFAULT NULL,  
  p_details TEXT DEFAULT NULL,  
  p_status TEXT DEFAULT 'SUCCESS',  
  p_ip TEXT DEFAULT NULL  
)  
RETURNS BIGINT  
LANGUAGE plpgsql  
AS $$  
DECLARE  
  v_id BIGINT;  
BEGIN  
  INSERT INTO audit_logs(user_id, action, resource, resource_id, details,  
                        status, ip_address, created_at)  
  VALUES (  
    p_user_id,  
    p_action,  
    p_resource,  
    p_res_id,  
    CASE WHEN p_details IS NOT NULL THEN p_details::JSONB ELSE NULL END,  
    p_status,  
    p_ip,  
    NOW()  
  )  
  RETURNING audit_id INTO v_id;  
  
  RETURN v_id;  
END;  
$$;
```

19. RecordUserLogin

Function_name: RecordUserLogin(p_user_id INT, p_ip TEXT DEFAULT NULL) → VOID

What it does: Updates users.last_login = NOW() and inserts a LOGIN entry into audit_logs with action='LOGIN', resource='users', and the IP address.

How it preserves consistency: Atomically stamps the login time and creates an audit record, ensuring last_login and audit_logs are always in sync.


```

-- ---- 8c. RecordUserLogin -----
-- Called by the auth layer on successful login.
-- Updates last_login on the user row and writes an audit entry.

CREATE FUNCTION RecordUserLogin(p_user_id INTEGER, p_ip TEXT DEFAULT NULL)
RETURNS VOID
LANGUAGE plpgsql
AS $$
BEGIN
    UPDATE users
    SET last_login = NOW()
    WHERE user_id = p_user_id;

    INSERT INTO audit_logs(user_id, action, resource, resource_id, ip_address, status, created_at)
    VALUES (p_user_id, 'LOGIN', 'users', p_user_id::TEXT, p_ip, 'SUCCESS', NOW());
END;
$$;

```

20. RecordUserLogout

```

-- ---- 8d. RecordUserLogout -----
-- Called by the auth layer when a session ends cleanly.

CREATE FUNCTION RecordUserLogout(p_user_id INTEGER)
RETURNS VOID
LANGUAGE plpgsql
AS $$
BEGIN
    INSERT INTO audit_logs(user_id, action, resource, resource_id, status, created_at)
    VALUES (p_user_id, 'LOGOUT', 'users', p_user_id::TEXT, 'SUCCESS', NOW());
END;
$$;

```

Function_name: RecordUserLogout(p_user_id INT) → VOID

What it does: Inserts a LOGOUT entry into audit_logs with action='LOGOUT', resource='users'.

How it preserves consistency: Maintains a complete session lifecycle in audit_logs (LOGIN → LOGOUT), supporting security monitoring and anomaly detection.

21. ReleaseLock

```
CREATE FUNCTION ReleaseLock(p_lock_id INTEGER)
RETURNS VOID
LANGUAGE plpgsql
AS $$
BEGIN
    DELETE FROM locks WHERE lock_id = p_lock_id;
END;
$$;
```

Function_name: ReleaseLock(p_lock_id INT) → VOID

What it does: Deletes the lock record from the locks table when a transaction releases its hold on a resource.

How it preserves consistency: Keeps the locks table reflecting only currently held locks, making active-lock queries accurate.

4.2 Triggers

The system implements 5 triggers that fire automatically on data changes, enforcing consistency rules at the database layer independently of the application.

1. trg_wallet_updated_at

```
-- Keeps wallet.last_updated current whenever the balance changes.
CREATE FUNCTION trg_fn_wallet_updated_at()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    NEW.last_updated := NOW();
    RETURN NEW;
END;
$$;

CREATE TRIGGER trg_wallet_updated_at
BEFORE UPDATE ON wallet
FOR EACH ROW
EXECUTE FUNCTION trg_fn_wallet_updated_at();
```

Event: BEFORE UPDATE ON wallet | FOR EACH ROW

What it does: Sets NEW.last_updated := NOW() on every update to a wallet row and returns NEW.

Consistency benefit: Ensures wallet.last_updated always reflects the true time of the last balance change, which is critical for audit trail accuracy and last-seen balance queries.

2. trg_users_updated_at

```
-- Keeps users.updated_at current on every update to that row.
CREATE FUNCTION trg_fn_users_updated_at()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    NEW.updated_at := NOW();
    RETURN NEW;
END;
$$;

CREATE TRIGGER trg_users_updated_at
    BEFORE UPDATE ON users
    FOR EACH ROW
    EXECUTE FUNCTION trg_fn_users_updated_at();
```

Event: BEFORE UPDATE ON users | FOR EACH ROW

What it does: Sets NEW.updated_at := NOW() on every update to a users row and returns NEW.

Consistency benefit: Guarantees users.updated_at is always accurate without requiring the application to pass the correct timestamp in every UPDATE statement.

3. trg_audit_user_update

```
-- Fires before any update to a users row.
-- Logs role changes and account activation / deactivation separately
-- so the audit trail clearly shows who changed what and when.
CREATE FUNCTION trg_fn_audit_user_update()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    IF OLD.role IS DISTINCT FROM NEW.role THEN
        INSERT INTO audit_logs(user_id, action, resource, resource_id, details, status)
        VALUES (
            NEW.user_id, 'USER_ROLE_CHANGED', 'users', NEW.user_id::TEXT,
            jsonb_build_object('old_role', OLD.role, 'new_role', NEW.role),
            'SUCCESS'
        );
    END IF;

    IF OLD.is_active IS DISTINCT FROM NEW.is_active THEN
        INSERT INTO audit_logs(user_id, action, resource, resource_id, details, status)
        VALUES (
            NEW.user_id,
            CASE WHEN NEW.is_active THEN 'USER_ACTIVATED' ELSE 'USER_DEACTIVATED' END,
            'users',
            NEW.user_id::TEXT,
            jsonb_build_object('is_active', NEW.is_active),
            'SUCCESS'
        );
    END IF;

    NEW.updated_at := NOW();
    RETURN NEW;
END;
$$;

CREATE TRIGGER trg_audit_user_update
    BEFORE UPDATE ON users
    FOR EACH ROW
    EXECUTE FUNCTION trg_fn_audit_user_update();
```

EVENT: BEFORE UPDATE ON users FOR EACH ROW

What it does: Checks two conditions using IS DISTINCT FROM (null-safe equality): (1) if role changed, inserts a USER_ROLE_CHANGED audit entry with old_role/new_role in JSNOB; (2) if is_active changed, inserts USER_ACTIVATED or USER_DEACTIVATED. Also stamps NEW.updated_at = NOW() before returning.

Consistency benefit: Role changes and account activations/deactivations are high-security events that must be immutably recorded. IS_DISTINCT FROM prevents false positives when unchanged fields appear in an UPDATE statement. Setting updated_at in the trigger guarantees the timestamp is always accurate.

4. trg_audit_wallet

```
-- Fires after every wallet_transactions insert (deposit or trade settlement).
CREATE FUNCTION trg_fn_audit_wallet()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
DECLARE
    v_user_id INTEGER;
BEGIN
    SELECT user_id INTO v_user_id FROM wallet WHERE wallet_id = NEW.wallet_id;

    INSERT INTO audit_logs(user_id, action, resource, resource_id, details, status, created_at)
    VALUES (
        v_user_id,
        'WALLET_' || NEW.transaction_type,
        'wallet',
        NEW.wallet_id::TEXT,
        jsonb_build_object(
            'transaction_id', NEW.transaction_id,
            'amount', NEW.amount,
            'transaction_type', NEW.transaction_type
        ),
        'SUCCESS',
        NOW()
    );
    RETURN NEW;
END;
$$;

CREATE TRIGGER trg_audit_wallet
AFTER INSERT ON wallet_transactions
FOR EACH ROW
EXECUTE FUNCTION trg_fn_audit_wallet();
```

Event: AFTER INSERT ON wallet_transactions | FOR EACH ROW

What it does: Resolves the wallet owner (user_id) from the wallet table, then inserts an audit_logs entry with action = 'WALLET_' || transaction_type (e.g., WALLET_CREDIT / WALLET_DEBIT) and a JSONB details object containing transaction_id, amount, and transaction_type.

Consistency benefit: Every fund movement is automatically logged to audit_logs, enabling financial reconciliation without relying on application-level logging. Resolving user_id from the wallet row ensures the record is tied to the correct user.

5. trg_audit_order

```
-- Fires after every new order and writes a record to audit_logs automatically.
-- This means the audit trail is always complete even if the app layer forgets.
CREATE FUNCTION trg_fn_audit_order()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    INSERT INTO audit_logs(user_id, action, resource, resource_id, details, status, created_at)
    VALUES (
        NEW.user_id,
        NEW.order_type || '_ORDER_' || NEW.order_status,
        'orders',
        NEW.order_id::TEXT,
        jsonb_build_object(
            'order_id', NEW.order_id,
            'stock_id', NEW.stock_id,
            'order_type', NEW.order_type,
            'quantity', NEW.quantity,
            'price', NEW.order_price,
            'total', NEW.order_price * NEW.quantity
        ),
        'SUCCESS',
        NOW()
    );
    RETURN NEW;
END;
$$;

CREATE TRIGGER trg_audit_order
AFTER INSERT ON orders
FOR EACH ROW
EXECUTE FUNCTION trg_fn_audit_order();
```

Event: AFTER INSERT ON orders | FOR EACH ROW

What it does: Inserts a row into audit_logs capturing user_id, a composite action string (order_type + '_ORDER_' + order_status), resource='orders', and a JSONB details object containing order_id, stock_id, order_type, quantity, price, and total value.

Consistency benefit: The audit trail is always complete, even if the application layer forgets to log. Firing AFTER INSERT guarantees the order row exists before the log entry is written.

5. ROLES and PRIVILGES

The system implements Role-Based Access Control (RBAC) through two layers: (a) PostgreSQL database-level roles granting structural access, and (b) application-level roles stored in the roles, permissions, and role_permissions tables governing fine-grained resource/action access.

5.1 Database-Level Roles

DB Role	Granted to	Priviledges
trade_flow_admin	DBAs/system admins	ALL PRIVILEGES on ALL TABLES, SEQUENCES, FUNCTIONS. Can CREATE / DROP objects.
trade_flow_user	Registered traders/investors	SELECT on stocks, stock_price_history, wallet (own), portfolio (own), orders (own), trades EXECUTE on PlaceOrder, CancelOrder, CreditWallet, DebitWallet, GetWalletBalance, GetPortfolioValue, GetProfitLoss, GetUserDashboard, GetUserOrderSummary, RecordUserLogin, RecordUserLogout
trade_flow_company	Company account holders	SELECT on company_profiles, company_announcements, stocks INSERT/UPDATE on company_announcements (own company) EXECUTE on WriteAuditLog, RecordUserLogin, RecordUserLogout

5.2 Application-Level Roles and Permissions

The roles table stores named roles (ADMIN, COMPANY, USER). The permissions table stores resource–action pairs. The role_permissions table maps roles to permitted actions.

USER	orders	create, cancel (own), read (own)
USER	portfolio	read (own)
USER	wallet	read (own), credit (deposit)
USER	stocks	read
USER	trades	read(own)
USER	company_announcements	read
ADMIN	users	read, write, delete,role_assign
ADMIN	orders	read(all users),cancel(any)
ADMIN	trades	read(all)
ADMIN	wallet	read(all), credit, debit
ADMIN	audit_logs	read, write
ADMIN	system_config	read, write
ADMIN	stocks	read, write, delete
ADMIN	portfolio	read(all)
COMPANY	company_profiles	read(own),wirte(own)
COMPANY	company_announcements	read, write, publish
COMPANY	stocks	read
COMPANY	orders	read(own stock only)

6. VIEWS

Views abstract complex joins and business logic into reusable, named queries. They also serve as a security layer — granting read access to a view rather than the underlying tables with sensitive columns.

1. view_dashboard

The **v_dashboard** view is designed to provide a comprehensive, user-level summary of financial and trading activity, consolidating wallet balance, portfolio performance, and order statistics into a single unified dashboard dataset.

```
CREATE VIEW public.v_dashboard AS
SELECT
  w.user_id,
  u.name AS user_name,
  ROUND(w.balance, 2) AS total_balance,
  COALESCE(port.total_invested, 0) AS total_invested,
  COALESCE(port.portfolio_value, 0) AS portfolio_value,
  COALESCE(port.portfolio_value, 0) - COALESCE(port.total_invested, 0) AS total_profit_loss,
  COALESCE(port.holdings_count, 0) AS holdings_count,
  COALESCE(ord.total_orders, 0) AS total_orders,
  COALESCE(ord.executed_orders, 0) AS executed_orders,
  COALESCE(ord.pending_orders, 0) AS pending_orders,
  COALESCE(ord.cancelled_orders, 0) AS cancelled_orders,
  w.last_updated AS wallet_updated_at
FROM public.wallet w
JOIN public.users u ON u.user_id = w.user_id
LEFT JOIN (
  SELECT
    p.user_id,
    COUNT(*)::INT AS holdings_count,
    ROUND(SUM(p.total_quantity * p.avg_buy_price), 2) AS total_invested,
    ROUND(SUM(p.total_quantity * COALESCE(lp.latest_price, p.avg_buy_price)), 2) AS portfolio_value
  FROM public.portfolio p
  LEFT JOIN public.v_stock_latest_price lp ON lp.stock_id = p.stock_id
  WHERE p.total_quantity > 0
  GROUP BY p.user_id
) port ON port.user_id = w.user_id
LEFT JOIN (
  SELECT
    user_id,
    COUNT(*)::INT AS total_orders,
    SUM(CASE WHEN upper(order_status) = 'EXECUTED' THEN 1 ELSE 0 END)::INT AS executed_orders,
    SUM(CASE WHEN upper(order_status) = 'PENDING' THEN 1 ELSE 0 END)::INT AS pending_orders,
    SUM(CASE WHEN upper(order_status) = 'CANCELLED' THEN 1 ELSE 0 END)::INT AS cancelled_orders
  FROM public.orders
  GROUP BY user_id
) ord ON ord.user_id = w.user_id;
```

Data Output Messages Notifications												
Showing rows: 1 to 102 Page No: 1 of 1												
	user_id integer	user_name character varying (100)	total_balance numeric	total_invested numeric	portfolio_value numeric	total_profit_loss numeric	holdings_count integer	total_orders integer	executed_orders integer	pending_orders integer	cancelled_orders integer	wallet_updated_at timestamp without time zone
1	1	Allison Hill	33400.38	68234.01	128855.98	60621.97	2	29	24	1	4	2026-04-07 17:57:15.996479
2	2	Angie Henderson	10406.47	190269.99	123696.49	-66573.50	3	20	10	8	2	2024-01-10 09:22:24
3	3	Cristian Santos	19488.47	1712073.39	1283454.04	-49519.25	13	137	53	72	12	2024-09-28 17:03:40
4	4	Abigail Shaffer	18735.52	710760.19	402107.64	-307652.55	6	54	21	28	5	2024-06-19 05:34:19
5	5	Neha Singh	29901.82	161665.47	168700.87	7035.40	2	26	5	19	2	2024-04-02 03:23:13
6	6	Lindsay Roman	39232.32	0	0	0	0	7	4	3	0	2024-12-13 20:57:17
7	7	Sandra Montgomery	34078.02	0	0	0	0	1	0	1	0	2024-09-28 07:22:51
8	8	Laura Bush	36852.97	0	0	0	0	9	1	2	0	2024-09-24 10:18:03
9	9	Sharon James	31982.00	92712.02	151124.64	58412.62	2	21	6	11	4	2024-02-19 19:08:05
10	10	Rahul Sharma	59816.75	322145.08	213174.40	-180970.66	2	4	3	1	0	2024-02-13 06:29:36
11	11	Kimberly Sanchez	32639.54	1603603.89	1313487.58	-340116.31	9	113	41	58	14	2024-12-21 07:33:16
12	12	Melinda Cameron	27031.25	76266.00	67456.80	-8809.20	1	6	3	2	1	2024-10-22 03:21:02
13	13	Austin Gentry	29310.83	2426.82	2548.20	380.62	1	9	1	6	2	2024-12-20 12:44:54
14	14	Derek Wright	2050.44	649331.09	704035.11	-145295.98	6	51	19	25	7	2024-03-12 14:39:25
15	15	Priya Mehta	32204.66	1109961.12	1067823.69	-61237.43	10	104	42	63	9	2024-07-22 08:10:43
16	16	Zachary Taylor	40568.33	0	0	0	0	1	0	0	1	2024-10-04 06:53:17
17	17	Christopher Becker	17841.38	10346.26	12813.98	246.72	1	9	4	3	2	2024-03-30 09:19:47
18	18	Tricia Valencia	23661.41	253369.29	212423.10	-42946.19	2	8	4	3	1	2024-11-10 06:15:09
19	19	Patricia Peterson	25495.33	8838.41	17959.04	9100.63	1	4	1	3	0	2024-08-07 20:55:06
20	20	Rahul Sharma	23956.02	209393.81	271332.83	61939.02	5	27	12	13	2	2024-09-30 15:30:01
21	21	Elizabeth Fowler	47774.75	97799.66	107557.90	9758.24	1	9	2	5	2	2024-09-21 14:46:28
22	22	Brittany Farmer	24412.02	290209.29	326278.79	30809.50	4	10	6	4	0	2024-09-08 00:07:35
23	23	Sherry Becker	3748.06	1107684.89	794289.48	-397295.41	10	132	50	70	12	2024-07-31 01:05:02
24	24	John Jones	7822.71	415828.40	369440.99	-46387.41	6	109	51	50	8	2024-09-06 09:52:47
25	25	Neha Singh	7293.27	1223797.19	926676.23	-297720.96	8	134	46	72	16	2024-12-06 05:47:21
26	26	Tanya Campos	42301.81	0	0	0	0	9	1	5	3	2024-08-10 13:02:30
27	27	Jessica Holmes	49645.75	99978.08	50487.25	-49490.82	1	6	1	5	0	2024-07-11 22:02:33
Total rows: 102 Query complete 00:00:00.133 CRLF Ln 1580, Col 1												

2. view_portfolio_detail

The **v_portfolio_detail** view is designed to provide a comprehensive analysis of a user's portfolio per holding, including investment value, current value, and profit/loss (P&L). It plays a critical role in portfolio tracking and financial analytics.

```
CREATE VIEW public.v_portfolio_detail AS
SELECT
    p.portfolio_id,
    p.user_id,
    p.stock_id,
    s.symbol,
    s.company_name,
    s.sector,
    p.total_quantity,
    p.avg_buy_price,
    COALESCE(lp.latest_price, p.avg_buy_price) AS current_price,
    ROUND(p.total_quantity * p.avg_buy_price, 2) AS total_invested,
    ROUND(p.total_quantity * p.avg_buy_price, 2) AS invested_value,
    ROUND(p.total_quantity * COALESCE(lp.latest_price, p.avg_buy_price), 2) AS current_value,
    ROUND(p.total_quantity * COALESCE(lp.latest_price, p.avg_buy_price), 2)
    - ROUND(p.total_quantity * p.avg_buy_price, 2) AS unrealized_pnl,
    CASE WHEN p.avg_buy_price > 0 THEN
        ROUND(((COALESCE(lp.latest_price, p.avg_buy_price) - p.avg_buy_price)
        / p.avg_buy_price) * 100, 2)
    ELSE 0 END AS pnl_pct
FROM public.portfolio p
JOIN public.stocks s ON s.stock_id = p.stock_id
LEFT JOIN public.v_stock_latest_price lp ON lp.stock_id = p.stock_id
WHERE p.total_quantity > 0;
```

```
1531 select * from v_portfolio_detail where portfolio_id=1
1532
1533
1534
```

portfolio_id	user_id	stock_id	symbol	company_name	sector	total_quantity	avg_buy_price	current_price	total_invested	invested_value	current_value	unrealized_pnl	pnl_pct	
1	1	1	6	LT	Larsen & Toubro	Engineeri...	47	1005.15	1436.51	47242.05	47242.05	67515.97	20273.92	42.91

3. view_company_stock_sentiment

The **v_company_stock_sentiment** view is designed to provide a comprehensive, aggregated analysis of stock trading behavior, enabling the system to derive insights into market sentiment for each stock.

```

CREATE OR REPLACE VIEW public.v_company_stock_sentiment AS
SELECT
  o.stock_id,
  s.symbol,
  s.company_name,
  COUNT(*) AS total_orders,
  SUM(CASE WHEN o.order_type = 'BUY' THEN 1 ELSE 0 END)::INT AS buy_count,
  SUM(CASE WHEN o.order_type = 'SELL' THEN 1 ELSE 0 END)::INT AS sell_count,
  SUM(CASE WHEN o.order_type = 'BUY' THEN o.quantity ELSE 0 END)::INT AS buy_volume,
  SUM(CASE WHEN o.order_type = 'SELL' THEN o.quantity ELSE 0 END)::INT AS sell_volume,
  ROUND(100.0 * SUM(CASE WHEN o.order_type = 'BUY' THEN 1 ELSE 0 END)
        / NULLIF(COUNT(*), 0), 1) AS buy_pct,
  ROUND(100.0 * SUM(CASE WHEN o.order_type = 'SELL' THEN 1 ELSE 0 END)
        / NULLIF(COUNT(*), 0), 1) AS sell_pct,
  (SELECT COUNT(DISTINCT user_id) FROM public.portfolio
   WHERE stock_id = o.stock_id AND total_quantity > 0)::INT AS total_holders
FROM public.orders o
JOIN public.stocks s ON s.stock_id = o.stock_id
WHERE upper(o.order_status) = 'EXECUTED'
GROUP BY o.stock_id, s.symbol, s.company_name;

```

	stock_id integer	symbol text	company_name text	total_orders bigint	buy_count integer	sell_count integer	buy_volume integer	sell_volume integer	buy_pct numeric	sell_pct numeric	total_holders integer
1	1	INFY	Infosys	61	40	21	1522	846	65.6	34.4	4
2	2	TCS	Tata Consultancy Servic...	48	25	23	1033	1028	52.1	47.9	4
3	3	RELIANCE	Reliance Industries	63	31	32	1097	1301	49.2	50.8	4
4	4	HDFCBANK	HDFC Bank	42	22	20	1012	1006	52.4	47.6	6
5	5	ICICIBANK	ICICI Bank	51	19	32	898	1328	37.3	62.7	6
6	6	LT	Larsen & Toubro	55	31	24	1446	804	56.4	43.6	6
7	7	ITC	ITC Limited	38	22	16	766	601	57.9	42.1	5
8	8	SBIN	State Bank of India	49	24	25	971	1160	49.0	51.0	2
9	9	WIPRO	Wipro	33	20	13	823	477	60.6	39.4	6
10	10	AXISBANK	Axie Bank	51	21	30	857	1204	41.2	58.8	6
11	11	MARUTI	Maruti Suzuki	70	36	34	1572	1325	51.4	48.6	8
12	12	TATAMOTO...	Tata Motors	52	30	22	1395	1164	57.7	42.3	4
13	13	ONGC	ONGC	56	28	28	1251	1032	50.0	50.0	7
14	14	NTPC	NTPC	46	23	23	988	899	50.0	50.0	5
15	15	ADANIENT	Adani Enterprises	57	31	26	1026	1091	54.4	45.6	7
16	16	ZOMATO	Zomato	50	26	24	1213	945	52.0	48.0	3
17	17	PAYTM	One97 Communications	57	37	20	1479	797	64.9	35.1	6
18	18	SWIGGY	Bundl Technologies	48	27	21	1257	824	56.3	43.6	5
19	19	BYJUS	Think & Learn	48	17	31	781	1433	35.4	64.6	1
20	20	NVKA	FSN E Commerce	46	22	24	758	1089	47.8	52.2	5

4. view_stock_latest_price

The **v_stock_latest_price** view is designed to provide the most recent market price for each stock, enabling accurate and real-time portfolio valuation and trading decisions.

```

378 -- Latest market price per stock -- used by portfolio valuation everywhere.
379 -- DISTINCT ON ensures we always get the most recent row per stock.
380 CREATE VIEW v_stock_latest_price AS
381 SELECT DISTINCT ON (sph.stock_id)
382     sph.stock_id,
383     s.symbol,
384     s.company_name,
385     s.sector,
386     sph.price AS latest_price,
387     sph.price_timestamp AS last_updated
388 FROM stock_price_history sph
389 JOIN stocks s ON s.stock_id = sph.stock_id
390 WHERE s.is_active = TRUE
391 ORDER BY sph.stock_id, sph.price_timestamp DESC;
392
393 -- From the view
394 SELECT * from v_stock_latest_price
395 WHERE stock_id = 1;
396
397

```

	stock_id integer	price numeric	price_timestamp timestamp without time zone
1	1	1877.81	2026-04-06 20:38:59.62755

5. view_order

The **v_orders** view is designed to provide a comprehensive and enriched representation of order data, combining order details with stock information to support order tracking, monitoring, and reporting.

```
-- Orders enriched with stock symbol and total value - used by the orders page.
CREATE VIEW v_orders AS
SELECT
    o.order_id,
    o.user_id,
    o.stock_id,
    s.symbol,
    s.company_name,
    s.sector,
    o.order_type,
    o.order_price,
    o.quantity,
    ROUND(o.order_price * o.quantity, 2) AS total_value,
    o.order_status,
    o.created_at
FROM orders o
JOIN stocks s ON s.stock_id = o.stock_id
ORDER BY o.created_at DESC;

select * from v_orders where order_id=1
```

Data Output Messages Notifications

Showing rows: 1 to 1 Page No: 1

	order_id integer	user_id integer	stock_id integer	symbol text	company_name text	sector text	order_type character varying	order_price numeric	quantity integer	total_value numeric	order_status character varying	created_at timestamp without time zone
1		1	11	12 TATAMOTO...	Tata Motors	Auto	SELL	2767.10	78	200830.30	PENDING	2024-11-07 17:59:02

6. view_trade

The **v_trades** view is designed to provide a detailed and enriched representation of executed trades, combining transactional, user, and stock information into a single unified dataset. This view is primarily used for displaying trade history in the system.


```

484 -- Trades enriched with user and stock details -- used by the trade history page.
485 CREATE VIEW v_trades AS
486 SELECT
487     t.trade_id,
488     t.order_id,
489     o.user_id,
490     o.stock_id,
491     s.symbol,
492     s.company_name,
493     o.order_type                AS trade_type,
494     t.executed_quantity         AS quantity,
495     t.executed_price            AS price,
496     ROUND(t.executed_price * t.executed_quantity, 2) AS total_value,
497     t.trade_time               AS executed_at
498 FROM trades t
499 JOIN orders o ON o.order_id = t.order_id
500 JOIN stocks s ON s.stock_id = o.stock_id
501 ORDER BY t.trade_time DESC;
502
503 select * from v_trades where trade_id=3
504
505
506
507

```

Data Output Messages Notifications

Showing rows: 1 to 1

	trade_id	order_id	user_id	stock_id	symbol	company_name	trade_type	quantity	price	total_value	executed_at
	integer	integer	integer	integer	text	text	character varying	integer	numeric	numeric	timestamp without time zone
1	3	2555	97	16	ZOMATO	Zomato	SELL	200	792.03	158406.00	2026-04-03 23:19:20.038264

7. view_admin_user_overview

The `view_admin_user_overview` is designed to provide a centralized and summarized snapshot of user activity in the system, specifically for administrative monitoring and decision-making.

```

CREATE OR REPLACE VIEW view_admin_user_overview AS
SELECT
    u.user_id,
    u.name,
    u.email,
    u.role,
    u.is_active,
    u.created_at,
    COALESCE(w.balance, 0)                AS wallet_balance,
    COALESCE(o.total_orders, 0)           AS total_orders,
    COALESCE(o.executed_orders, 0)        AS executed_orders,
    COALESCE(o.pending_orders, 0)         AS pending_orders,
    COALESCE(p.holdings_count, 0)         AS holdings_count
FROM users u
LEFT JOIN wallet w ON w.user_id = u.user_id
LEFT JOIN (
    SELECT
        user_id,
        COUNT(*)                AS total_orders,
        SUM(CASE WHEN upper(order_status) = 'EXECUTED' THEN 1 ELSE 0 END) AS executed_orders,
        SUM(CASE WHEN upper(order_status) = 'PENDING' THEN 1 ELSE 0 END) AS pending_orders
    FROM orders
    GROUP BY user_id
) o ON o.user_id = u.user_id
LEFT JOIN (
    SELECT
        user_id,
        COUNT(DISTINCT stock_id) AS holdings_count
    FROM portfolio
    WHERE total_quantity > 0
    GROUP BY user_id
) p ON p.user_id = u.user_id;

```


8. view_company_stock_sentiment

The `v_company_stock_sentiment` view is designed to provide a comprehensive, aggregated analysis of stock trading behavior, enabling the system to derive market sentiment insights for each stock.

```
CREATE OR REPLACE VIEW public.v_company_stock_sentiment AS
SELECT
  o.stock_id,
  s.symbol,
  s.company_name,
  COUNT(*) AS total_orders,
  SUM(CASE WHEN o.order_type = 'BUY' THEN 1 ELSE 0 END)::INT AS buy_count,
  SUM(CASE WHEN o.order_type = 'SELL' THEN 1 ELSE 0 END)::INT AS sell_count,
  SUM(CASE WHEN o.order_type = 'BUY' THEN o.quantity ELSE 0 END)::INT AS buy_volume,
  SUM(CASE WHEN o.order_type = 'SELL' THEN o.quantity ELSE 0 END)::INT AS sell_volume,
  ROUND(100.0 * SUM(CASE WHEN o.order_type = 'BUY' THEN 1 ELSE 0 END)
        / NULLIF(COUNT(*), 0), 1) AS buy_pct,
  ROUND(100.0 * SUM(CASE WHEN o.order_type = 'SELL' THEN 1 ELSE 0 END)
        / NULLIF(COUNT(*), 0), 1) AS sell_pct,
  (SELECT COUNT(DISTINCT user_id) FROM public.portfolio
   WHERE stock_id = o.stock_id AND total_quantity > 0)::INT AS total_holders
FROM public.orders o
JOIN public.stocks s ON s.stock_id = o.stock_id
WHERE upper(o.order_status) = 'EXECUTED'
GROUP BY o.stock_id, s.symbol, s.company_name;
```

7. ADDITIONAL INDICES

1. Composite

```
-- =====
-- COMPOSITE (multi-column)
-- =====
CREATE INDEX idx_portfolio_user_stock ON portfolio(user_id, stock_id);
CREATE INDEX idx_orders_user_status ON orders(user_id, order_status);
CREATE INDEX idx_sph_stock_timestamp ON stock_price_history(stock_id, price_timestamp DESC);
```

a. idx_portfolio_user_stock

Purpose: To speed up queries that retrieve a user's holdings for a specific stock using `user_id` and `stock_id`.

Justification:

- This is a multi-column index optimized for queries filtering by:
 - `user_id`
 - and `stock_id` together

b. idx_orders_user_status

Purpose: To optimize filtering of orders based on `user_id` and `order_status` for fast retrieval of user activity and order statistics.

Justification

- Optimizes queries filtering orders by:
 - user
 - and status (EXECUTED, PENDING, CANCELLED)

c. idx_sph_stock_timestamp

Purpose: To efficiently fetch the latest stock price for each stock by supporting sorting on `price_timestamp` in descending order.

Justification

- Designed specifically for retrieving latest stock price

2. HASH INDEX

a. idx_audit_details_gin

Purpose: This index is created to optimize exact-match lookups on user email, which is a very common operation in authentication and user management.

```
-- =====  
-- HASH (exact equality only)  
-- =====  
CREATE INDEX idx_users_email_hash ON users USING HASH(email);
```

3. GIN

```
-- =====  
-- GIN (JSONB search inside details column)  
-- =====  
CREATE INDEX idx_audit_details_gin ON audit_logs USING GIN(details);
```

a. idx_audit_details_gin

Purpose: To enable efficient searching inside the `details` JSONB column of the `audit_logs` table.

Justification

- The `details` column is of type **JSONB**, which stores semi-structured data.
- Traditional B-Tree indexes are not efficient for JSON searches.

Frequent Queries that Use these Indices

1. Portfolio with P&L (Core Feature)

```
SELECT symbol, total_quantity, avg_buy_price,  
current_price, current_value, unrealized_pnl, pnl_pct  
FROM fn_get_user_portfolio(1)  
ORDER BY unrealized_pnl DESC;
```

2. Dashboard Summary (System Integration)

```
SELECT wallet_balance, total_invested, portfolio_value, total_orders,  
executed_orders  
FROM v_dashboard  
WHERE user_id = 1;
```

3. Stock Sentiment Analysis (Analytics)

```
SELECT total_orders, buy_count, sell_count,  
buy_pct, sell_pct, total_holders
```

```
FROM v_company_stock_sentiment  
WHERE stock_id = 1;
```

4. Top Gainers (Advanced Logic + Time Series)

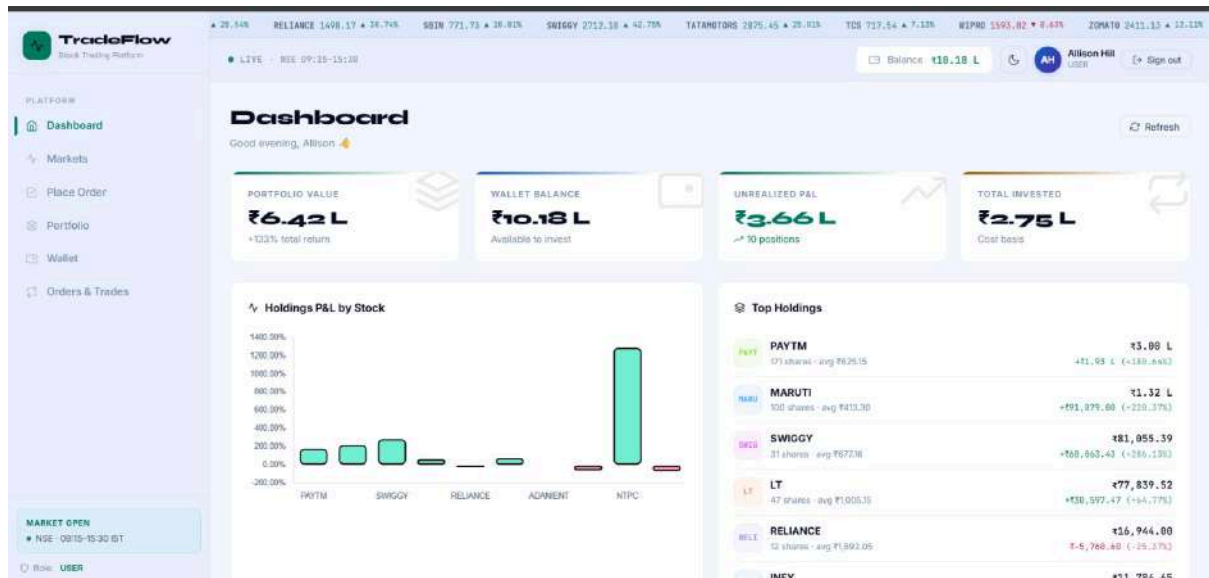
```
SELECT symbol, company_name, current_price, prev_price, change_pct  
FROM fn_top_gainers(5);
```

5. Audit Log Search Where used has walled_debit(can be used for determining sell operations (Index + JSONB Usage))

```
SELECT user_id, action, resource, details, created_at  
FROM audit_logs  
WHERE action = 'WALLET_DEBIT'  
ORDER BY created_at DESC  
LIMIT 50;
```

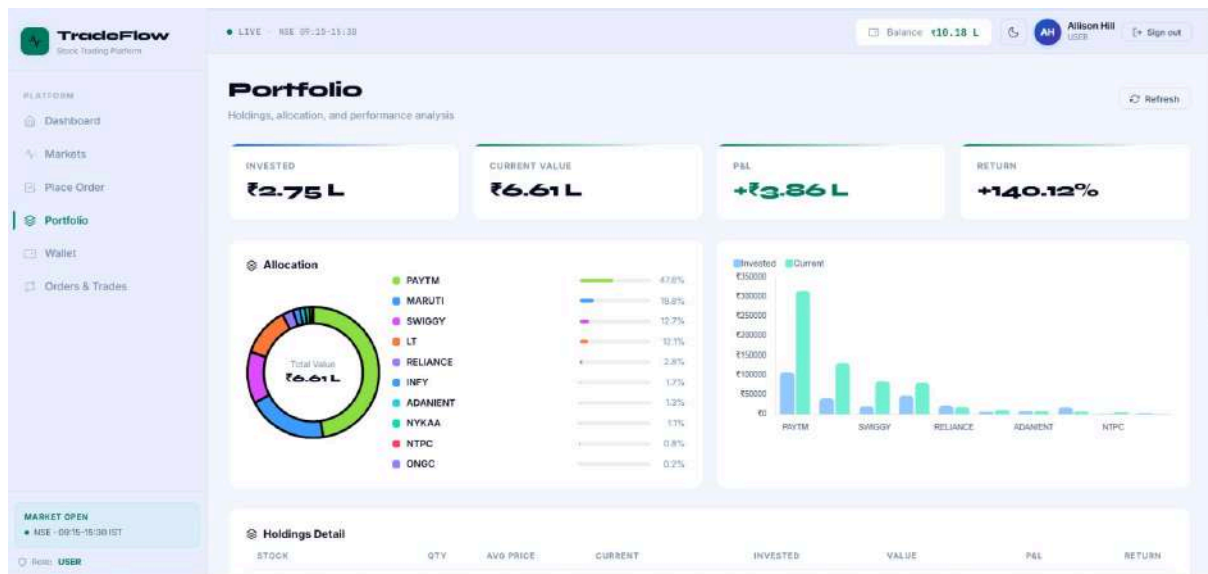
8. FRONTEND

A] USER DASHBOARD



This screen displays the overall financial summary of the user, including portfolio value, wallet balance, unrealized profit/loss, and total invested amount. The data is primarily fetched using functions such as `getuserdashboard()`, `getportfoliovalue()`, `getprofitloss()`, `getwalletbalance()`, and `getwalletsummary()`. These functions aggregate data from multiple tables, like portfolio, wallet, and trades, to provide real-time insights. Additionally, the latest stock prices used in charts and holdings are retrieved using the `v_stock_latest_price` view. Background triggers such as `trg_fn_wallet_updated_at` ensure that timestamps and related data remain consistent when updates occur.

B] USER PORTFOLIO



This screen presents detailed information about the user's holdings, including allocation, current value, invested amount, profit/loss, and returns. The system uses functions like `getuserdashboard()`, `getportfoliovalue()`, and `getprofitloss()` to compute these values dynamically. The `updateportfolio()` function ensures that portfolio data is updated whenever trades are executed. The `v_stock_latest_price` view is used to fetch current market prices for accurate valuation. Triggers such as `trg_fn_wallet_updated_at` and audit-related triggers help maintain consistency and log updates to the system.

C] PLACE ORDER = BUY

The screenshot shows the 'Place Order' interface of the TracleFlow Stock Trading Platform. The interface is divided into several sections:

- Top Header:** Displays the platform name 'TracleFlow' and a list of stock prices for various companies like WIPRO, ZOMATO, ADANI, etc.
- Left Sidebar:** Contains navigation links for 'Dashboard', 'Markets', 'Place Order' (active), 'Portfolio', 'Wallet', and 'Orders & Trades'.
- Main Content Area:**
 - Place Order:** A section with a subtitle 'Execute trades with real-time validation and atomic settlement'. It features a 'BUY' button (green) and a 'SELL' button (red).
 - STOCK:** A dropdown menu showing 'NTPC - NTPC'.
 - ORDER TYPE:** Three buttons: 'MARKET' (selected), 'LIMIT', and 'SL'.
 - QUANTITY:** A text input field containing '10'.
 - MARKET PRICE (₹):** A text input field containing '2556.16'.
 - Auto-Bid:** A note stating 'Auto-Bid from live market price'.
 - Buy NTPC:** A green button with the text 'Buy NTPC ₹25,561.60'.
- Market Details:** A section on the right showing real-time data for NTPC:
 - Current Price: ₹2,556.16
 - Prev Close: ₹2,135.02
 - Today's Change: +19.73%
 - Sector: Power
 - Your Holdings: 2 shares · avg ₹125.84
- Order Summary:** A section on the right showing the order details:
 - Shares: 10
 - Price/share: ₹2,556.16
 - Total: ₹25,561.60
 - Wallet usage: 2.5%
 - Netted: ₹25,561.60
 - Margin: ₹63.90 L

This screen allows users to place buy orders for stocks. When a user initiates a buy action, the `placeorder()` function is triggered, which validates and records the order. It then calls `executeorder()` to process the order and `createtrade()` to record the completed transaction. The wallet balance is reduced using `debitwallet()`, and the portfolio is updated through `updateportfolio()`. Functions like `getwalletbalance()` and `getuserdashboard()` are used to display real-time balance and holdings. The `v_stock_latest_price` view provides the current market price. Triggers such as `trg_fn_audit_order`, `trg_fn_audit_wallet`, and `trg_fn_wallet_updated_at` ensure auditing and consistency.

D] PLACE ORDER = SELL

The screenshot shows the 'Place Order' page in the TradeFlow Stock Trading Platform. The interface is designed for executing a sell order. On the left, a sidebar lists navigation options: Dashboard, Markets, Place Order (selected), Portfolio, Wallet, and Orders & Trades. The main content area is titled 'Place Order' with the subtitle 'Execute trades with real-time validation and atomic settlement'. It features a 'BUY' button (disabled) and a 'SELL' button (active). Below these, the 'STOCK' is set to 'NTPC - NTPC'. The 'ORDER TYPE' is set to 'MARKET'. The 'QUANTITY' is set to '10'. The 'MARKET PRICE (₹)' is displayed as '2529.78'. A red button at the bottom says 'Sell NTPC - ₹25,297.80'. To the right, the 'Market Details' section shows the current price at ₹2,529.78, a pre-close of ₹2,135.02, and a today's change of +18.49%. Below this, the 'Order Summary' shows 10 shares at ₹2,529.78 per share, totaling ₹25,297.80. A warning message states 'Insufficient shares'.

This screen facilitates the selling of stocks from the user's portfolio. Similar to the buy flow, the `placeorder()` function is invoked, followed by `executeorder()` and `createtrade()` to complete the transaction. The user's wallet is updated using `creditwallet()` after selling, and portfolio quantities are adjusted via `updateportfolio()`. The system also uses `getwalletbalance()` and `getuserdashboard()` to reflect updated financial data. Market prices are fetched through the `v_stock_latest_price` view. Audit and consistency are maintained using triggers such as `trg_fn_audit_order`, `trg_fn_audit_wallet`, and `trg_fn_wallet_updated_at`.

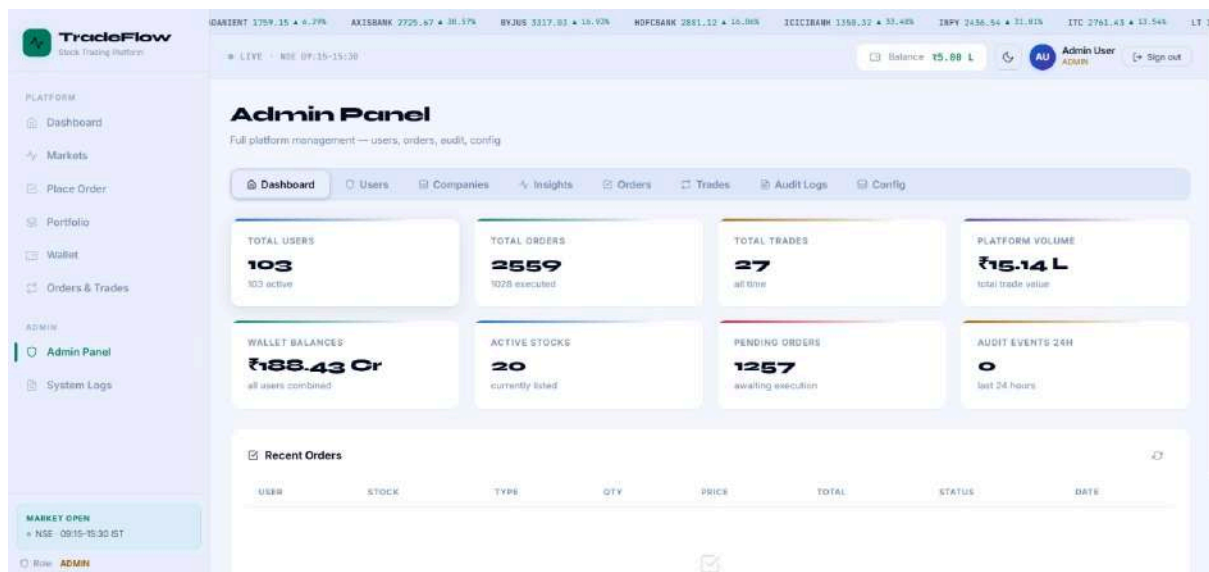
E] RECORDS OF ORDERS AND TRADES OF THE USER

The screenshot shows the 'Orders & Trades' page in the TradeFlow Stock Trading Platform. The interface displays a summary of orders and trades. At the top, there are four boxes showing the counts: TOTAL (31), EXECUTED (26), PENDING (1), and CANCELLED (4). Below these, there are tabs for 'Orders' and 'Trades'. The 'Order Book' section is active, showing a table of orders. The table has columns for ID, SYMBOL, TYPE, PRICE TYPE, QTY, PRICE, TOTAL, STATUS, and DATE. The orders are listed in descending order of date.

ID	SYMBOL	TYPE	PRICE TYPE	QTY	PRICE	TOTAL	STATUS	DATE
1582	ADANIENT	SELL	MARKET	5	₹-	₹-	✓ EXECUTED	18/04/2026, 15:37:30
1581	ADANIENT	BUY	MARKET	10	₹-	₹-	✓ EXECUTED	18/04/2026, 15:37:12
1577	RELIANCE	BUY	MARKET	2	₹-	₹-	✓ EXECUTED	07/04/2026, 15:37:05
1576	MARUTI	SELL	MARKET	2	₹-	₹-	✓ EXECUTED	07/04/2026, 12:23:47
1575	MARUTI	BUY	MARKET	2	₹-	₹-	✓ EXECUTED	07/04/2026, 12:23:42
1573	PAYTM	BUY	MARKET	1	₹-	₹-	✓ EXECUTED	05/04/2026, 12:51:09
1570	MARUTI	BUY	MARKET	100	₹-	₹-	✓ EXECUTED	04/04/2026, 23:06:54

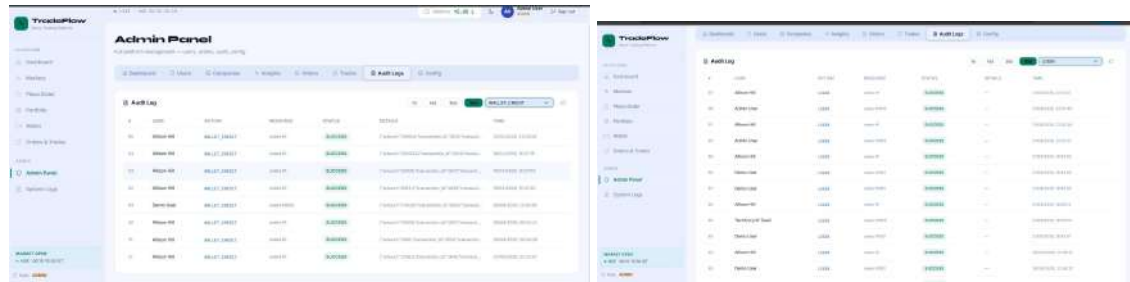
This screen displays all orders placed by the user along with their execution status. Functions such as `getuserorderssummary()` are used to fetch summarized order data, while `placeorder()`, `executeorder()`, and `createtrade()` manage the lifecycle of each order. The `cancelorder()` function allows users to cancel pending orders. The data is directly retrieved from orders and trades tables without relying on views. Triggers like `trg_fn_audit_order` and `trg_fn_audit_wallet` log changes and ensure consistency in order and transaction records.

F] ADMIN DASHBOARD



This screen provides a high-level overview of platform activity for administrators, including total users, orders, trades, and trading volume. The data is aggregated using the `getadminstats()` function, which collects information from multiple tables across the system. The `getadminuserprofile()` function may be used for user-level insights. Triggers such as `trg_fn_audit_order`, `trg_fn_audit_wallet`, and `trg_fn_audit_user_update` ensure that all system changes are tracked and logged properly for auditing purposes.

G] ADMIN AUDIT LOGS = [LOGIN, CREDIT_WALLET, DEBIT_WALLET ,etc.]



ID	User	Action	Status	Timestamp
1	Admin	Login	Success	2023-03-24 10:00:00
2	Admin	Credit Wallet	Success	2023-03-24 10:05:00
3	Admin	Debit Wallet	Success	2023-03-24 10:10:00
4	Admin	Update User	Success	2023-03-24 10:15:00
5	Admin	Logout	Success	2023-03-24 10:20:00

This screen displays the audit logs of critical system activities including user logins, wallet credit transactions, and wallet debit operations. The data is generated and recorded using functions such as `writeauditlog()` and `logoperation()`, which capture detailed information about each event, including user, action, status, and timestamp. Wallet-related activities are handled through functions like `creditwallet()` and `debitwallet()`, which automatically trigger logging mechanisms. The system relies on triggers such as `trg_fn_audit_wallet`, `trg_fn_audit_order`, and `trg_fn_audit_user_update` to ensure that all significant operations are recorded without manual intervention. Additionally, `trg_fn_wallet_updated_at` maintains consistency by updating timestamps during wallet modifications. The audit logs are retrieved directly from the audit logs table, allowing administrators to monitor system activity and ensure transparency and accountability.

H] COMPANY DASHBOARD



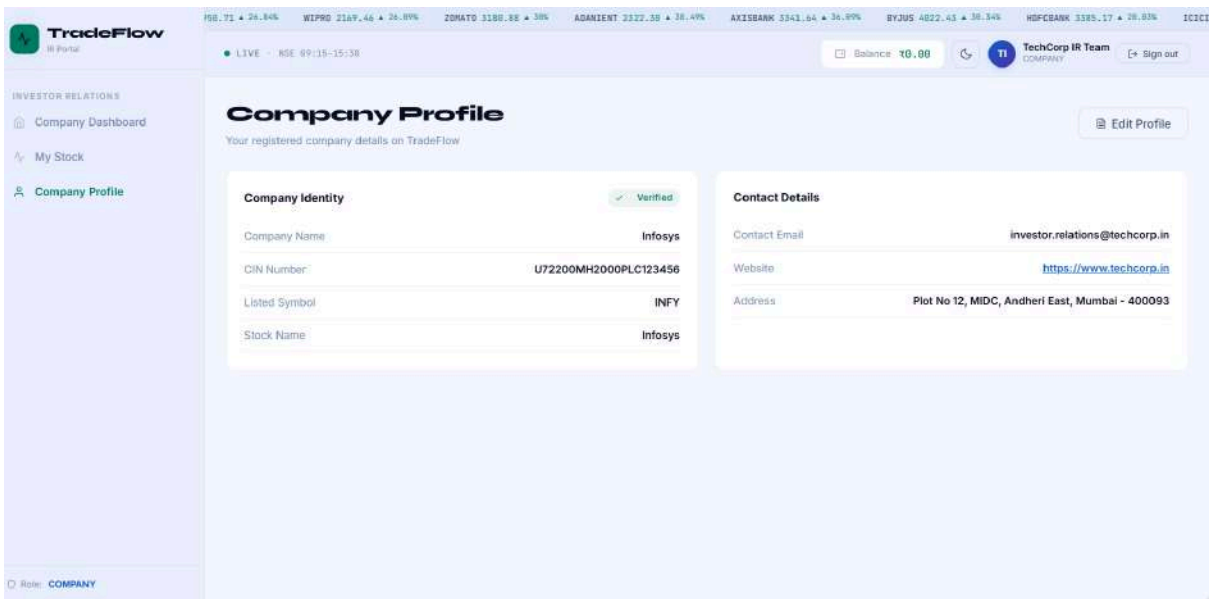
The company dashboard utilizes functions like `getadminstats()` and partially `getstockstats()` to display metrics such as current price, total investors, and trading volume. These values are derived from trades and holdings data. The triggers `trg_fn_audit_order` and `trg_fn_audit_trade` ensure that all trading activity affecting these metrics is recorded. Views like `v_stock_latest_price` are used to fetch the current price and historical trends.

I] COMPANY STOCK



This screen is powered by the `getstockstats()` function, which provides detailed statistics for a specific stock including price trends, investor count, and sentiment analysis. The triggers `trg_fn_audit_trade` and `trg_fn_audit_order` ensure accurate tracking of trading activity. Views such as `v_stock_latest_price` and `v_trade_summary` are used to simplify data retrieval for charts and sentiment calculations.

J] COMPANY PROFILE



This screen is powered primarily by the `getcompanyprofile()` function, which fetches all company-related details such as company name, CIN, stock symbol, and contact information. When the user edits the profile, the `updatecompanyprofile` function is triggered to update the respective fields in the database. To maintain accountability and track changes, the `trg_fn_audit_user_update` trigger is activated whenever any modification is made, ensuring that all updates are recorded in the audit logs. The data displayed on this screen is likely retrieved through the `v_company_details` view, which simplifies data access by combining information from multiple related tables.

9. CONCLUSION

The TradeFlow database is a comprehensive, production-ready relational schema that demonstrates the application of core DBMS principles to a real-world financial domain. The design achieves strong data integrity through layered constraints (primary keys, foreign keys, check, not null, unique, and defaults), automates critical business rules through stored functions and triggers, enforces principle-of-least-privilege access via a two-tier RBAC system, simplifies complex queries and hardens security through well-justified views, and ensures high query performance through a carefully chosen set of additional indices beyond the default primary-key indices. Together these features ensure that the system can reliably handle concurrent trading activity, maintain a complete audit trail, and support efficient analytics — all fundamental requirements of a financial trading platform.